RGA IM2D API 开发指南

文件标识：RK-KF-YF-403

发布版本：V2.2.7

日期：2025-04-30

文件密级：☐绝密 ☐秘密 ☐内部资料 ■公开

免责声明

本文档按"现状"提供，瑞芯微电子股份有限公司（"本公司"，下同）不对本文档的任何陈述、信息和内容的准确性、可靠性、完整性、适销性、特定目的性和非侵权性提供任何明示或暗示的声明或保证。本文档仅作为使用指导的参考。

由于产品版本升级或其他原因，本文档将可能在未经任何通知的情况下，不定期进行更新或修改。

商标声明

"Rockchip"、"瑞芯微"、"瑞芯"均为本公司的注册商标，归本公司所有。

本文档可能提及的其他所有注册商标或商标，由其各自拥有者所有。

版权所有 **© 2022** 瑞芯微电子股份有限公司

超越合理使用范畴，非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

瑞芯微电子股份有限公司

Rockchip Electronics Co., Ltd.

地址： 福建省福州市铜盘路软件园A区18号

网址： www.rock-chips.com

客户服务电话： +86-4007-700-590

客户服务传真： +86-591-83951833

客户服务邮箱： fae@rock-chips.com

读者对象

本文档（本指南）主要适用于以下工程师：

- 技术支持工程师
- 软件开发工程师

修订记录

日期	版本	作者	修改说明
2020/06/24	1.0.0	陈城，李煌	初始版本
2020/10/16	1.0.1	陈城，李煌，余乔伟	更新部分接口
2021/12/07	2.0.0	陈城，李煌，余乔伟	增加RGA3相关支持
2022/01/20	2.1.0	陈城，李煌，余乔伟	- 更新im2d api接口说明 - 更新硬件指标说明，以及对齐限制 - 增加数据结构介绍
2022/06/22	2.1.1	陈城，李煌，余乔伟	完善格式支持/对齐说明
2022/09/15	2.2.0	陈城，李煌，余乔伟	- 补充默认值相关说明 - 新增array接口 - 新增task接口 - 新增矩形边框绘制接口
2023/02/09	2.2.1	余乔伟	更正文档格式
2023/06/28	2.2.2	余乔伟	- 增加芯片RK3562介绍 - 完善针对灰度图的注意事项
2024/03/06	2.2.3	余乔伟	增加芯片RK3576介绍
2024/08/22	2.2.4	余乔伟	增加芯片RK3506、RV1103B介绍
2024/11/18	2.2.5	余乔伟	增加API在不同环境支持情况
2025/03/27	2.2.6	余乔伟	增加芯片R1126B介绍
2025/04/30	2.2.7	余乔伟	更正RK3506、RV1103B描述中对实高的过度约束

目 录

RGA IM2D API 开发指南
1. 概述
1.1 设计指标
1.2 图像格式支持
1.3 图像格式对齐说明
2. 版本说明
2.1 librga API版本说明
2.1.1 版本号格式与递增规则
2.1.1.1 版本号格式
2.1.1.2 递增规则
2.1.2 版本号查询
2.1.2.1 strings命令查询:
2.1.2.2 日志打印:
2.1.2.3 函数接口查询
2.1.2.4 属性查询
2.2 驱动版本说明
2.2.1 版本号格式与递增规则
2.2.1.1 版本号格式
2.2.1.2 递增规则
2.2.2 版本号查询
2.2.2.1 开机日志查询:
2.2.2.2 调试节点查询
2.3 版本对应关系
3. 应用接口
3.1 概述
3.2 获取RGA 版本及支持信息
3.2.1 querystring
3.3 头文件版本校验
3.3.1 imcheckHeader
3.4 图像缓冲区预处理
3.4.1 importbuffer_T
3.4.2 releasebuffer_handle
3.4.3 wrapbuffer_handle
3.5 图像处理任务创建
3.5.1 imbeginJob
3.6 图像处理任务提交
3.6.1 imendJob
3.7 图像处理任务取消
3.7.1 imcancelJob
3.8 图像拷贝
3.8.1 imcopy
3.8.2 imcopyTask
3.9 图像缩放、图像金字塔
3.9.1 imresize
3.9.2 impyramid
3.9.3 imresizeTask
3.10 图像裁剪
3.10.1 imcrop
3.10.2 imcropTask
3.11 图像平移
3.11.1 imtranslate
3.11.2 imtranslateTask
3.12 图像格式转换
3.12.1 imcvtcolor
3.12.2 imcvtcolorTask
3.13 图像旋转
3.13.1 imrotate
3.13.2 imrotateTask
3.14 图像镜像翻转
3.14.1 imfilp
3.14.2 imflipTask
3.15 图像合成
3.15.1 imblend/imcomposite
3.15.2 imblendTask/imcompositeTask

3.16 色键（Color Key）
3.16.1 imcolorkey
3.16.2 imcolorkeyTask
3.17 图像字幕叠加（OSD）
3.17.1 imosd
3.17.2 imosdTask
3.18 NN运算点前处理（量化）
3.18.1 imquantize
3.18.2 imquantizeTask
3.19 图像光栅操作 ROP
3.19.1 imrop
3.19.2 imropTask
3.20 图像颜色填充、边框绘制
3.20.1 imfill
3.20.2 imfillArray
3.20.3 imfillTask
3.20.4 imfillTaskArray
3.20.5 imrectangle
3.20.6 imrectangleArray
3.20.7 imrectangleTask
3.20.8 imrectangleTaskArray
3.20.9 immakeBorder
3.21 图像马赛克
3.21.1 immosaic
3.21.2 immosaicArray
3.21.3 immosaicTask
3.21.4 immosaicTaskArray
3.22 图像处理
3.22.1 improcess
3.22.2 improcessTask
3.23 参数校验
3.23.1 imcheck
3.24 同步操作
3.24.1 imsync
3.25 线程上下文配置
3.25.1 imconfig
4. 数据结构
4.1 概述
4.2 详细描述
4.2.1 rga_buffer_t
4.2.2 im_rect
4.2.3 im_opt_t
4.2.4 im_job_handle_t
4.2.5 rga_buffer_handle_t
4.2.6 im_handle_param_t
4.2.7 im_nn_t
4.2.8 im_colorkey_range
4.2.9 im_osd_block_t
4.2.10 im_osd_invert_factor_t
4.2.11 im_osd_invert_t
4.2.12 im_osd_bpp2_t
4.2.13 im_osd_t
5. 测试用例及调试方法
5.1 测试文件说明
5.2 调试方法说明
5.3 测试用例说明
5.3.1 申请图像缓冲
5.3.1.1 Graphicbuffer
5.3.1.2 AHardwareBuffer
5.3.2 查看帮助信息
5.3.3 循环执行demo
5.3.4 获取RGA版本及支持信息
5.3.4.1 代码解析
5.3.5 图像缩放
5.3.5.1 代码解析
5.3.6 图像裁剪
5.3.6.1 代码解析

5.3.7 图像旋转
5.3.7.1 代码解析
5.3.8 图像镜像翻转
5.3.8.1 代码解析
5.3.9 图像颜色填充
5.3.9.1 代码解析
5.3.10 图像平移
5.3.10.1 代码解析
5.3.11 图像拷贝
5.3.11.1 代码解析
5.3.12 图像合成
5.3.12.1 代码解析
5.3.13 图像格式转换
5.3.13.1 代码解析

1. 概述

RGA (Raster Graphic Acceleration Unit)是一个独立的2D硬件加速器，可用于加速点/线绘制，执行图像缩放、旋转、bitBlt、alpha混合等常见的2D图形操作。

1.1 设计指标

Version	Codename	Chip	Source		Destination		Function	Pixels/C
			min	max	min	max		
RGA1	Pagani	RK3066	2x2	8192x8192	2x2	2048x2048	90/180/270 Rotate X/Y Mirror Crop 1/2~8 scale Alpha blend Color key Color fill ROP IOMMU(32bit)	1
	Jaguar Plus	RK3188						
	Beetles	RK2926/2928						
	Beetles Plus	RK3026/3028						
RGA1_plus	Audi	RK3128	2x2	8192x8192	2x2	2048x2048	90/180/270 Rotate X/Y Mirror Crop 1/2~8 scale Alpha blend Color key Color fill Color palette IOMMU(32bit)	1
	Granite	Sofia 3gr						
RGA2	Lincoln	RK3288/3288w	2x2	8192x8192	2x2	4096x4096	90/180/270 Rotate X/Y Mirror Crop 1/16~16 scale scale-up(bi-linear/bi-cubic) scale-down(average) Alpha blend Color key Color fill Color palette ROP IOMMU(32bit)	2
	Capricorn	RK3190						
RGA2-Lite0	Maybach	RK3368	2x2	8192x8192	2x2	4096x4096	90/180/270 Rotate X/Y Mirror Crop 1/8~8 scale scale-up(bi-linear/bi-cubic) scale-down(average) Alpha blend Color key Color fill Color palette ROP IOMMU(32bit)	2
	BMW	RK3366						

RGA2-Lite1	Benz	RK3228	2x2	8192x8192	2x2	4096x4096	90/180/270 Rotate X/Y Mirror Crop 1/8~8 scale scale-up(bi-linear/bi-cubic) scale-down(average) Alpha blend Color key Color fill Color palette IOMMU(32bit)	2
	Infiniti	RK3228H						
	Gemini	RK3326						
	Lion	RK1808						
RGA2-Lite2	Libra	RK3506	2x2	1280x8192	2x2	1280x4096	90/180/270 Rotate X/Y Mirror Crop 1/16~16 scale scale-up(bi-linear/bi-cubic) scale-down(average) Alpha blend Color key Color fill Color palette Guassion blur alpha-8bit	2
RGA2-Lite3	Pather	RK1103B	2x2	2880x8192	2x2	2880x8192	90/180/270 Rotate X/Y Mirror Crop 1/16~16 scale scale-up(bi-linear) scale-down(bi-linear/average) Color fill	2
RGA2-Enhance	Mclaren	RK3399	2x2	8192x8192	2x2	4096x4096	90/180/270 Rotate X/Y Mirror Crop 1/16~16 scale scale-up(bi-linear/bi-cubic) scale-down(average) Alpha blend Color key Color fill Color palette ROP(NA for RV1108/RV1109/RK3566) NN quantize(NA for RK3399/RV1108) osd (only RV1106/RV1103/RK3562/RK3528) mosaic(only RV1106/RV1103/RK3562/RK3528) IOMMU(32bit, RK3528/RK3562为40bit，NA for RV1106/1103)	2
	Mercury	RK1108						
	Puma	RV1126/RV1109						
	skylarkV2	RK3566/RK3568						
	Orion	RK3588						
	Otter	RV1106/1103						
	Bull	RK3528						
	Snipe	RK3562						

RGA2-Pro	Heron	RK3576	2x2	8192x8192	2x2	8192x8192	90/180/270 Rotate X/Y Mirror Crop 1/16~16 scale scale-up(bi-linear/bi-cubic) scale-down(bi-linear/average) Alpha blend Color key Color fill Color palette ROP osd mosaic(only RK3576) ARGB5551 alpha bit map rkfbc64x4 input(only RK3576) afbc32x8 input(split mode, only RK3576) tile4x4(RV1126B only input) IOMMU(40bit)	2
	Swan	RV1126B						
RGA3	Orion	RK3588	68x2	8176x8176	68x2	8128x8128	90/180/270 Rotate X/Y Mirror Crop 1/8~8 scale scale-up(bi-cubic) scale-down(average) Alpha blend Color key afbc16x16 IOMMU(40bit)	3 (by pa 2 (scale)

> 注：
>
> 1). 单位时钟周期处理像素的能力为理论数据，实际运行性能表现与带宽、硬件频率等相关，列表数据仅供参考。
>
> 2). 除最小输入分辨率限制外，每个通道可设置的实际操作矩形的x、y、width、height参数必须大于等于2。
>
> 3). RGA的寻址能力和IOMMU的bit位数是相关联的，例如搭载支持32bit IOMMU的RGA实际的物理地址寻址能力仅支持0~4G的内存空间。

1.2 图像格式支持

- Pixel Format conversion, BT.601/BT.709/BT.2020(only RGA3)
- Dither operation

<table>
<tr><th>Version</th><th>Codename</th><th>Chip</th><th>Input Data Format</th><th>Output Data Format</th></tr>
<tr><td rowspan="4">RGA1</td><td>Pagani</td><td>RK3066</td><td rowspan="4">RK_FORMAT_RGBA_8888
RK_FORMAT_BGRA_8888
RK_FORMAT_ARGB_8888
RK_FORMAT_ABGR_8888
RK_FORMAT_RGBX_8888
RK_FORMAT_BGRX_8888
RK_FORMAT_XRGB_8888
RK_FORMAT_XBGR_8888

RK_FORMAT_ARGB_4444
RK_FORMAT_ABGR_4444
RK_FORMAT_ARGB_5551
RK_FORMAT_ABGR_5551
RK_FORMAT_RGB_888
RK_FORMAT_BGR_888
RK_FORMAT_RGB_565
RK_FORMAT_BGR_565
RK_FORMAT_YCbCr_420_SP
RK_FORMAT_YCrCb_420_SP
RK_FORMAT_YCbCr_422_SP
RK_FORMAT_YCrCb_422_SP
RK_FORMAT_YCbCr_420_P
RK_FORMAT_YCrCb_420_P
RK_FORMAT_YCbCr_422_P
RK_FORMAT_YCrCb_422_P
RK_FORMAT_BPP1
RK_FORMAT_BPP2
RK_FORMAT_BPP4
RK_FORMAT_BPP8</td><td rowspan="4">RK_FORMAT_RGBA_8888
RK_FORMAT_BGRA_8888
RK_FORMAT_ARGB_8888
RK_FORMAT_ABGR_8888
RK_FORMAT_RGBX_8888
RK_FORMAT_BGRX_8888
RK_FORMAT_XRGB_8888
RK_FORMAT_XBGR_8888
RK_FORMAT_ARGB_4444
RK_FORMAT_ABGR_4444
RK_FORMAT_ARGB_5551
RK_FORMAT_ABGR_5551
RK_FORMAT_RGB_888
RK_FORMAT_BGR_888
RK_FORMAT_RGB_565
RK_FORMAT_BGR_565
RK_FORMAT_YCbCr_420_SP (only for Blur/sharpness)
RK_FORMAT_YCrCb_420_SP (only for Blur/sharpness)
RK_FORMAT_YCbCr_422_SP (only for Blur/sharpness)
RK_FORMAT_YCrCb_422_SP (only for Blur/sharpness)
RK_FORMAT_YCbCr_420_P (only for Blur/sharpness)
RK_FORMAT_YCrCb_420_P (only for Blur/sharpness)
RK_FORMAT_YCbCr_422_P (only for Blur/sharpness)
RK_FORMAT_YCrCb_422_P (only for Blur/sharpness)</td></tr>
<tr><td>Jaguar Plus</td><td>RK3188</td></tr>
<tr><td>Beetles</td><td>RK2926/2928</td></tr>
<tr><td>Beetles Plus</td><td>RK3026/3028</td></tr>
</table>

RGA1_plus	Audi	RK3128	RK_FORMAT_RGBA_8888 RK_FORMAT_BGRA_8888 RK_FORMAT_ARGB_8888 RK_FORMAT_ABGR_8888 RK_FORMAT_RGBX_8888 RK_FORMAT_BGRX_8888 RK_FORMAT_XRGB_8888 RK_FORMAT_XBGR_8888 RK_FORMAT_ARGB_4444 RK_FORMAT_ABGR_4444 RK_FORMAT_ARGB_5551 RK_FORMAT_ABGR_5551 RK_FORMAT_RGB_888 RK_FORMAT_BGR_888 RK_FORMAT_RGB_565 RK_FORMAT_BGR_565 RK_FORMAT_YCbCr_420_SP RK_FORMAT_YCrCb_420_SP RK_FORMAT_YCbCr_422_SP RK_FORMAT_YCrCb_422_SP RK_FORMAT_YCbCr_420_P RK_FORMAT_YCrCb_420_P RK_FORMAT_YCbCr_422_P RK_FORMAT_YCrCb_422_P RK_FORMAT_BPP1 RK_FORMAT_BPP2 RK_FORMAT_BPP4 RK_FORMAT_BPP8	RK_FORMAT_RGBA_8888 RK_FORMAT_BGRA_8888 RK_FORMAT_ARGB_8888 RK_FORMAT_ABGR_8888 RK_FORMAT_RGBX_8888 RK_FORMAT_BGRX_8888 RK_FORMAT_XRGB_8888 RK_FORMAT_XBGR_8888 RK_FORMAT_ARGB_4444 RK_FORMAT_ABGR_4444 RK_FORMAT_ARGB_5551 RK_FORMAT_ABGR_5551 RK_FORMAT_RGB_888 RK_FORMAT_BGR_888 RK_FORMAT_RGB_565 RK_FORMAT_BGR_565 RK_FORMAT_YCbCr_420_SP (only for normal Bitblt without alpha) RK_FORMAT_YCrCb_420_SP (only for normal Bitblt without alpha) RK_FORMAT_YCbCr_422_SP (only for normal Bitblt without alpha) RK_FORMAT_YCrCb_422_SP (only for normal Bitblt without alpha) RK_FORMAT_YCbCr_420_P (only for normal Bitblt without alpha) RK_FORMAT_YCrCb_420_P (only for normal Bitblt without alpha) RK_FORMAT_YCbCr_422_P (only for normal Bitblt without alpha) RK_FORMAT_YCrCb_422_P (only for normal Bitblt without alpha)
	Granite	Sofia 3gr		

RGA2	Lincoln	RK3288/3288w	RK_FORMAT_RGBA_8888 RK_FORMAT_BGRA_8888 RK_FORMAT_ARGB_8888 RK_FORMAT_ABGR_8888 RK_FORMAT_RGBX_8888 RK_FORMAT_BGRX_8888 RK_FORMAT_XRGB_8888 RK_FORMAT_XBGR_8888 RK_FORMAT_ARGB_4444 RK_FORMAT_ABGR_4444 RK_FORMAT_ARGB_5551 RK_FORMAT_ABGR_5551 RK_FORMAT_RGB_888 RK_FORMAT_BGR_888 RK_FORMAT_RGB_565 RK_FORMAT_BGR_565 RK_FORMAT_YCbCr_420_SP RK_FORMAT_YCrCb_420_SP RK_FORMAT_YCbCr_422_SP RK_FORMAT_YCrCb_422_SP RK_FORMAT_YCbCr_420_P RK_FORMAT_YCrCb_420_P RK_FORMAT_YCbCr_422_P RK_FORMAT_YCrCb_422_P RK_FORMAT_BPP1 (only for color palette) RK_FORMAT_BPP2 (only for color palette) RK_FORMAT_BPP4 (only for color palette) RK_FORMAT_BPP8 (only for color palette)	RK_FORMAT_RGBA_8888 RK_FORMAT_BGRA_8888 RK_FORMAT_ARGB_8888 RK_FORMAT_ABGR_8888 RK_FORMAT_RGBX_8888 RK_FORMAT_BGRX_8888 RK_FORMAT_XRGB_8888 RK_FORMAT_XBGR_8888 RK_FORMAT_ARGB_4444 RK_FORMAT_ABGR_4444 RK_FORMAT_ARGB_5551 RK_FORMAT_ABGR_5551 RK_FORMAT_RGB_888 RK_FORMAT_BGR_888 RK_FORMAT_RGB_565 RK_FORMAT_BGR_565 RK_FORMAT_YCbCr_420_SP RK_FORMAT_YCrCb_420_SP RK_FORMAT_YCbCr_422_SP RK_FORMAT_YCrCb_422_SP RK_FORMAT_YCbCr_420_P RK_FORMAT_YCrCb_420_P RK_FORMAT_YCbCr_422_P RK_FORMAT_YCrCb_422_P
	Capricorn	RK3190		

RGA2-Lite0	Maybach	RK3368	RK_FORMAT_RGBA_8888 RK_FORMAT_BGRA_8888 RK_FORMAT_ARGB_8888 RK_FORMAT_ABGR_8888 RK_FORMAT_RGBX_8888 RK_FORMAT_BGRX_8888 RK_FORMAT_XRGB_8888 RK_FORMAT_XBGR_8888 RK_FORMAT_ARGB_4444 RK_FORMAT_ABGR_4444 RK_FORMAT_ARGB_5551 RK_FORMAT_ABGR_5551 RK_FORMAT_RGB_888 RK_FORMAT_BGR_888 RK_FORMAT_RGB_565 RK_FORMAT_BGR_565 RK_FORMAT_YCbCr_420_SP RK_FORMAT_YCrCb_420_SP RK_FORMAT_YCbCr_422_SP RK_FORMAT_YCrCb_422_SP RK_FORMAT_YCbCr_420_P RK_FORMAT_YCrCb_420_P RK_FORMAT_YCbCr_422_P RK_FORMAT_YCrCb_422_P RK_FORMAT_BPP1 (only for color palette) RK_FORMAT_BPP2 (only for color palette) RK_FORMAT_BPP4 (only for color palette) RK_FORMAT_BPP8 (only for color palette)	RK_FORMAT_RGBA_8888 RK_FORMAT_BGRA_8888 RK_FORMAT_ARGB_8888 RK_FORMAT_ABGR_8888 RK_FORMAT_RGBX_8888 RK_FORMAT_BGRX_8888 RK_FORMAT_XRGB_8888 RK_FORMAT_XBGR_8888 RK_FORMAT_ARGB_4444 RK_FORMAT_ABGR_4444 RK_FORMAT_ARGB_5551 RK_FORMAT_ABGR_5551 RK_FORMAT_RGB_888 RK_FORMAT_BGR_888 RK_FORMAT_RGB_565 RK_FORMAT_BGR_565 RK_FORMAT_YCbCr_420_SP RK_FORMAT_YCrCb_420_SP RK_FORMAT_YCbCr_422_SP RK_FORMAT_YCrCb_422_SP RK_FORMAT_YCbCr_420_P RK_FORMAT_YCrCb_420_P RK_FORMAT_YCbCr_422_P RK_FORMAT_YCrCb_422_P
	BMW	RK3366		

RGA2-Lite1	Benz	RK3228	RK_FORMAT_RGBA_8888 RK_FORMAT_BGRA_8888 RK_FORMAT_ARGB_8888 RK_FORMAT_ABGR_8888 RK_FORMAT_RGBX_8888 RK_FORMAT_BGRX_8888 RK_FORMAT_XRGB_8888 RK_FORMAT_XBGR_8888 RK_FORMAT_RGBA_4444 RK_FORMAT_ARGB_5551 RK_FORMAT_ABGR_5551 RK_FORMAT_RGB_888 RK_FORMAT_BGR_888 RK_FORMAT_RGB_565 RK_FORMAT_BGR_565 RK_FORMAT_YCbCr_420_SP RK_FORMAT_YCrCb_420_SP RK_FORMAT_YCbCr_422_SP RK_FORMAT_YCrCb_422_SP RK_FORMAT_YCbCr_420_P RK_FORMAT_YCrCb_420_P RK_FORMAT_YCbCr_422_P RK_FORMAT_YCrCb_422_P RK_FORMAT_YCbCr_420_SP_10B RK_FORMAT_YCrCb_420_SP_10B RK_FORMAT_YCbCr_422_SP_10B RK_FORMAT_YCrCb_422_SP_10B RK_FORMAT_BPP1 (only for color palette) RK_FORMAT_BPP2 (only for color palette) RK_FORMAT_BPP4 (only for color palette) RK_FORMAT_BPP8 (only for color palette)	RK_FORMAT_RGBA_8888 RK_FORMAT_BGRA_8888 RK_FORMAT_ARGB_8888 RK_FORMAT_ABGR_8888 RK_FORMAT_RGBX_8888 RK_FORMAT_BGRX_8888 RK_FORMAT_XRGB_8888 RK_FORMAT_XBGR_8888 RK_FORMAT_ARGB_4444 RK_FORMAT_ABGR_4444 RK_FORMAT_ARGB_5551 RK_FORMAT_ABGR_5551 RK_FORMAT_RGB_888 RK_FORMAT_BGR_888 RK_FORMAT_RGB_565 RK_FORMAT_BGR_565 RK_FORMAT_YCbCr_420_SP RK_FORMAT_YCrCb_420_SP RK_FORMAT_YCbCr_422_SP RK_FORMAT_YCrCb_422_SP RK_FORMAT_YCbCr_420_P RK_FORMAT_YCrCb_420_P RK_FORMAT_YCbCr_422_P RK_FORMAT_YCrCb_422_P
	Infiniti	RK3228H		
	Gemini	RK3326		
	Lion	RK1808		

RGA2-Lite2	Libra	RK3506	RK_FORMAT_RGBA_8888 RK_FORMAT_BGRA_8888 RK_FORMAT_ARGB_8888 RK_FORMAT_ABGR_8888 RK_FORMAT_RGBX_8888 RK_FORMAT_BGRX_8888 RK_FORMAT_XRGB_8888 RK_FORMAT_XBGR_8888 RK_FORMAT_ARGB_4444 RK_FORMAT_ABGR_4444 RK_FORMAT_ARGB_5551 RK_FORMAT_ABGR_5551 RK_FORMAT_RGB_888 RK_FORMAT_BGR_888 RK_FORMAT_RGB_565 RK_FORMAT_BGR_565 RK_FORMAT_YCbCr_420_SP RK_FORMAT_YCrCb_420_SP RK_FORMAT_YCbCr_422_SP RK_FORMAT_YCrCb_422_SP RK_FORMAT_YCbCr_420_P RK_FORMAT_YCrCb_420_P RK_FORMAT_YCbCr_422_P RK_FORMAT_YCrCb_422_P RK_FORMAT_YUYV_422 RK_FORMAT_YVYU_422 RK_FORMAT_UYVY_422 RK_FORMAT_VYUY_422 RK_FORMAT_YCbCr_400 RK_FORMAT_YCbCr_420_SP_10B RK_FORMAT_YCrCb_420_SP_10B RK_FORMAT_YCbCr_422_SP_10B RK_FORMAT_YCrCb_422_SP_10B RK_FORMAT_A8 RK_FORMAT_BPP1 (only for color palette) RK_FORMAT_BPP2 (only for color palette) RK_FORMAT_BPP4 (only for color palette) RK_FORMAT_BPP8 (only for color palette)	RK_FORMAT_RGBA_8888 RK_FORMAT_BGRA_8888 RK_FORMAT_ARGB_8888 RK_FORMAT_ABGR_8888 RK_FORMAT_RGBX_8888 RK_FORMAT_BGRX_8888 RK_FORMAT_XRGB_8888 RK_FORMAT_XBGR_8888 RK_FORMAT_ARGB_4444 RK_FORMAT_ABGR_4444 RK_FORMAT_ARGB_5551 RK_FORMAT_ABGR_5551 RK_FORMAT_RGB_888 RK_FORMAT_BGR_888 RK_FORMAT_RGB_565 RK_FORMAT_BGR_565 RK_FORMAT_YCbCr_420_SP RK_FORMAT_YCrCb_420_SP RK_FORMAT_YCbCr_422_SP RK_FORMAT_YCrCb_422_SP RK_FORMAT_YCbCr_420_P RK_FORMAT_YCrCb_420_P RK_FORMAT_YCbCr_422_P RK_FORMAT_YCrCb_422_P RK_FORMAT_YUYV_422 RK_FORMAT_YVYU_422 RK_FORMAT_UYVY_422 RK_FORMAT_VYUY_422 RK_FORMAT_YCbCr_400 RK_FORMAT_Y4

RGA2-Lite2	Pather	RV1103B	RK_FORMAT_RGBX_8888 RK_FORMAT_BGRX_8888 RK_FORMAT_XRGB_8888 RK_FORMAT_XBGR_8888 RK_FORMAT_RGB_888 RK_FORMAT_BGR_888 RK_FORMAT_RGB_565 RK_FORMAT_BGR_565 RK_FORMAT_YCbCr_420_SP RK_FORMAT_YCrCb_420_SP RK_FORMAT_YCbCr_422_SP RK_FORMAT_YCrCb_422_SP RK_FORMAT_YCbCr_420_P RK_FORMAT_YCrCb_420_P RK_FORMAT_YCbCr_422_P RK_FORMAT_YCrCb_422_P RK_FORMAT_YUYV_422 RK_FORMAT_YVYU_422 RK_FORMAT_UYVY_422 RK_FORMAT_VYUY_422 RK_FORMAT_YCbCr_400	RK_FORMAT_RGBX_8888 RK_FORMAT_BGRX_8888 RK_FORMAT_XRGB_8888 RK_FORMAT_XBGR_8888 RK_FORMAT_RGB_888 RK_FORMAT_BGR_888 RK_FORMAT_RGB_565 RK_FORMAT_BGR_565 RK_FORMAT_YCbCr_420_SP RK_FORMAT_YCrCb_420_SP RK_FORMAT_YCbCr_422_SP RK_FORMAT_YCrCb_422_SP RK_FORMAT_YCbCr_420_P RK_FORMAT_YCrCb_420_P RK_FORMAT_YCbCr_422_P RK_FORMAT_YCrCb_422_P RK_FORMAT_YUYV_422 RK_FORMAT_YVYU_422 RK_FORMAT_UYVY_422 RK_FORMAT_VYUY_422 RK_FORMAT_YCbCr_400

RGA2-Enhance	Mclaren	RK3399	RK_FORMAT_RGBA_8888 RK_FORMAT_BGRA_8888 RK_FORMAT_ARGB_8888 RK_FORMAT_ABGR_8888 RK_FORMAT_RGBX_8888 RK_FORMAT_BGRX_8888 RK_FORMAT_XRGB_8888 RK_FORMAT_XBGR_8888 RK_FORMAT_ARGB_4444 RK_FORMAT_ABGR_4444 RK_FORMAT_ARGB_5551 RK_FORMAT_ABGR_5551 RK_FORMAT_RGB_888 RK_FORMAT_BGR_888 RK_FORMAT_RGB_565 RK_FORMAT_BGR_565 RK_FORMAT_YCbCr_420_SP RK_FORMAT_YCrCb_420_SP RK_FORMAT_YCbCr_422_SP RK_FORMAT_YCrCb_422_SP RK_FORMAT_YCbCr_420_P RK_FORMAT_YCrCb_420_P RK_FORMAT_YCbCr_422_P RK_FORMAT_YCrCb_422_P RK_FORMAT_YCbCr_420_SP_10B RK_FORMAT_YCrCb_420_SP_10B RK_FORMAT_YCbCr_422_SP_10B RK_FORMAT_YCrCb_422_SP_10B RK_FORMAT_BPP1 (only for color palette) RK_FORMAT_BPP2 (only for color palette) RK_FORMAT_BPP4 (only for color palette) RK_FORMAT_BPP8 (only for color palette)	RK_FORMAT_RGBA_8888 RK_FORMAT_BGRA_8888 RK_FORMAT_ARGB_8888 RK_FORMAT_ABGR_8888 RK_FORMAT_RGBX_8888 RK_FORMAT_BGRX_8888 RK_FORMAT_XRGB_8888 RK_FORMAT_XBGR_8888 RK_FORMAT_ARGB_4444 RK_FORMAT_ABGR_4444 RK_FORMAT_ARGB_5551 RK_FORMAT_ABGR_5551 RK_FORMAT_RGB_888 RK_FORMAT_BGR_888 RK_FORMAT_RGB_565 RK_FORMAT_BGR_565 RK_FORMAT_YCbCr_420_SP RK_FORMAT_YCrCb_420_SP RK_FORMAT_YCbCr_422_SP RK_FORMAT_YCrCb_422_SP RK_FORMAT_YCbCr_420_P RK_FORMAT_YCrCb_420_P RK_FORMAT_YCbCr_422_P RK_FORMAT_YCrCb_422_P RK_FORMAT_YUYV_422 RK_FORMAT_YVYU_422 RK_FORMAT_UYVY_422 RK_FORMAT_VYUY_422
	Mercury	RK1108		

	Puma	RV1126/ RV1109	RK_FORMAT_RGBA_8888 RK_FORMAT_BGRA_8888 RK_FORMAT_ARGB_8888 RK_FORMAT_ABGR_8888 RK_FORMAT_RGBX_8888 RK_FORMAT_BGRX_8888 RK_FORMAT_XRGB_8888 RK_FORMAT_XBGR_8888 RK_FORMAT_ARGB_4444 RK_FORMAT_ABGR_4444 RK_FORMAT_ARGB_5551 RK_FORMAT_ABGR_5551 RK_FORMAT_RGB_888 RK_FORMAT_BGR_888 RK_FORMAT_RGB_565 RK_FORMAT_BGR_565 RK_FORMAT_YCbCr_420_SP RK_FORMAT_YCrCb_420_SP RK_FORMAT_YCbCr_422_SP RK_FORMAT_YCrCb_422_SP RK_FORMAT_YCbCr_420_P RK_FORMAT_YCrCb_420_P RK_FORMAT_YCbCr_422_P RK_FORMAT_YCrCb_422_P RK_FORMAT_YUYV_422 RK_FORMAT_YVYU_422 RK_FORMAT_UYVY_422 RK_FORMAT_VYUY_422 RK_FORMAT_YCbCr_400 RK_FORMAT_YCbCr_420_SP_10B RK_FORMAT_YCrCb_420_SP_10B RK_FORMAT_YCbCr_422_SP_10B RK_FORMAT_YCrCb_422_SP_10B RK_FORMAT_BPP1 (only for color palette) RK_FORMAT_BPP2 (only for color palette) RK_FORMAT_BPP4 (only for color palette) RK_FORMAT_BPP8 (only for color palette)	RK_FORMAT_RGBA_8888 RK_FORMAT_BGRA_8888 RK_FORMAT_ARGB_8888 RK_FORMAT_ABGR_8888 RK_FORMAT_RGBX_8888 RK_FORMAT_BGRX_8888 RK_FORMAT_XRGB_8888 RK_FORMAT_XBGR_8888 RK_FORMAT_ARGB_4444 RK_FORMAT_ABGR_4444 RK_FORMAT_ARGB_5551 RK_FORMAT_ABGR_5551 RK_FORMAT_RGB_888 RK_FORMAT_BGR_888 RK_FORMAT_RGB_565 RK_FORMAT_BGR_565 RK_FORMAT_YCbCr_420_SP RK_FORMAT_YCrCb_420_SP RK_FORMAT_YCbCr_422_SP RK_FORMAT_YCrCb_422_SP RK_FORMAT_YCbCr_420_P RK_FORMAT_YCrCb_420_P RK_FORMAT_YCbCr_422_P RK_FORMAT_YCrCb_422_P RK_FORMAT_YUYV_422 RK_FORMAT_YVYU_422 RK_FORMAT_UYVY_422 RK_FORMAT_VYUY_422 RK_FORMAT_YCbCr_400 RK_FORMAT_Y4
	skylarkV2	RK3566/RK3568		
	Orion	RK3588		
	Otter	RV1106/1103		
	Bull	RK3528		
	Snipe	RK3562		

RGA2-Pro	Heron	RK3576	RK_FORMAT_RGBA_8888 RK_FORMAT_BGRA_8888 RK_FORMAT_ARGB_8888 RK_FORMAT_ABGR_8888 RK_FORMAT_RGBX_8888 RK_FORMAT_BGRX_8888 RK_FORMAT_XRGB_8888 RK_FORMAT_XBGR_8888 RK_FORMAT_ARGB_4444 RK_FORMAT_ABGR_4444 RK_FORMAT_ARGB_5551 RK_FORMAT_ABGR_5551 RK_FORMAT_RGB_888 RK_FORMAT_BGR_888 RK_FORMAT_RGB_565 RK_FORMAT_BGR_565 RK_FORMAT_YCbCr_420_SP RK_FORMAT_YCrCb_420_SP RK_FORMAT_YCbCr_422_SP RK_FORMAT_YCrCb_422_SP RK_FORMAT_YCbCr_444_SP RK_FORMAT_YCrCb_444_SP RK_FORMAT_YCbCr_420_P RK_FORMAT_YCrCb_420_P RK_FORMAT_YCbCr_422_P RK_FORMAT_YCrCb_422_P RK_FORMAT_YUYV_422 RK_FORMAT_YVYU_422 RK_FORMAT_UYVY_422 RK_FORMAT_VYUY_422 RK_FORMAT_YCbCr_400 RK_FORMAT_YCbCr_420_SP_10B RK_FORMAT_YCrCb_420_SP_10B RK_FORMAT_YCbCr_422_SP_10B RK_FORMAT_YCrCb_422_SP_10B RK_FORMA_A8 (only src for alpha blend) RK_FORMAT_BPP1 (only for color palette) RK_FORMAT_BPP2 (only for color palette) RK_FORMAT_BPP4 (only for color palette) RK_FORMAT_BPP8 (only for color palette)	RK_FORMAT_RGBA_8888 RK_FORMAT_BGRA_8888 RK_FORMAT_ARGB_8888 RK_FORMAT_ABGR_8888 RK_FORMAT_RGBX_8888 RK_FORMAT_BGRX_8888 RK_FORMAT_XRGB_8888 RK_FORMAT_XBGR_8888 RK_FORMAT_ARGB_4444 RK_FORMAT_ABGR_4444 RK_FORMAT_ARGB_5551 RK_FORMAT_ABGR_5551 RK_FORMAT_RGB_888 RK_FORMAT_BGR_888 RK_FORMAT_RGB_565 RK_FORMAT_BGR_565 RK_FORMAT_YCbCr_420_SP RK_FORMAT_YCrCb_420_SP RK_FORMAT_YCbCr_422_SP RK_FORMAT_YCrCb_422_SP RK_FORMAT_YCbCr_444_SP RK_FORMAT_YCrCb_444_SP RK_FORMAT_YCbCr_420_P RK_FORMAT_YCrCb_420_P RK_FORMAT_YCbCr_422_P RK_FORMAT_YCrCb_422_P RK_FORMAT_YUYV_422 RK_FORMAT_YVYU_422 RK_FORMAT_UYVY_422 RK_FORMAT_VYUY_422 RK_FORMAT_YCbCr_400 RK_FORMAT_Y4 (only RK3576) RK_FORMAT_Y8 (only RK3576)
	Swan	RV1126B		

RGA3	Orion	RK3588	RK_FORMAT_RGBA_8888 RK_FORMAT_BGRA_8888 RK_FORMAT_ARGB_8888 RK_FORMAT_ABGR_8888 RK_FORMAT_RGBX_8888 RK_FORMAT_BGRX_8888 RK_FORMAT_XRGB_8888 RK_FORMAT_XBGR_8888 RK_FORMAT_RGB_888 RK_FORMAT_BGR_888 RK_FORMAT_RGB_565 RK_FORMAT_BGR_565 RK_FORMAT_YCbCr_420_SP RK_FORMAT_YCrCb_420_SP RK_FORMAT_YCbCr_422_SP RK_FORMAT_YCrCb_422_SP RK_FORMAT_YUYV_422 RK_FORMAT_YVYU_422 RK_FORMAT_UYVY_422 RK_FORMAT_VYUY_422 RK_FORMAT_YCbCr_420_SP_10B RK_FORMAT_YCrCb_420_SP_10B RK_FORMAT_YCbCr_422_SP_10B RK_FORMAT_YCrCb_422_SP_10B	RK_FORMAT_RGBA_8888 RK_FORMAT_BGRA_8888 RK_FORMAT_RGBX_8888 RK_FORMAT_BGRX_8888 RK_FORMAT_RGB_888 RK_FORMAT_BGR_888 RK_FORMAT_RGB_565 RK_FORMAT_BGR_565 RK_FORMAT_YCbCr_420_SP RK_FORMAT_YCrCb_420_SP RK_FORMAT_YCbCr_422_SP RK_FORMAT_YCrCb_422_SP RK_FORMAT_YUYV_422 RK_FORMAT_YVYU_422 RK_FORMAT_UYVY_422 RK_FORMAT_VYUY_422 RK_FORMAT_YCbCr_420_SP_10B RK_FORMAT_YCrCb_420_SP_10B RK_FORMAT_YCbCr_422_SP_10B RK_FORMAT_YCrCb_422_SP_10B

> 注：
>
> 1). ”RK_FORMAT_YCbCr_400“格式即YUV格式仅取Y通道，常用于256（2的8次方）阶灰度图，这里需要注意由于是YUV格式存在RGB/YUV色域转换时需要留意色域配置，例如需要完整的256阶灰度图需要在转换时配置为full range。
>
> 2). “RK_FORMAT_Y4”格式即YUV格式仅取Y通道，并dither至4bit，常用于16（2的4次方）阶灰度图，涉及色域转换时配置的注意事项同“RK_FORMAT_YCbCr_400”。“RK_FORMAT_Y8”格式与“RK_FORMAT_Y4”类似，同样只有4bit有效，差异在于只是其中高4bit为有效数据，低4bit为无效数据。

1.3 图像格式对齐说明

Version	Byte_stride	Format	Alignment
RGA1 RGA1-Plus	4	RK_FORMAT_RGBA_8888 RK_FORMAT_BGRA_8888 RK_FORMAT_ARGB_8888 RK_FORMAT_ABGR_8888 RK_FORMAT_RGBX_8888 RK_FORMAT_BGRX_8888 RK_FORMAT_XRGB_8888 RK_FORMAT_XBGR_8888	width stride无对齐要求
		RK_FORMAT_RGBA_4444 RK_FORMAT_BGRA_4444 RK_FORMAT_ARGB_4444 RK_FORMAT_ABGR_4444 RK_FORMAT_RGBA_5551 RK_FORMAT_BGRA_5551 RK_FORMAT_ARGB_5551 RK_FORMAT_ABGR_5551 RK_FORMAT_RGB_565 RK_FORMAT_BGR_565	width stride须2对齐
		RK_FORMAT_RGB_888 RK_FORMAT_BGR_888	width stride须4对齐
		RK_FORMAT_YCbCr_420_SP RK_FORMAT_YCrCb_420_SP RK_FORMAT_YCbCr_422_SP RK_FORMAT_YCrCb_422_SP RK_FORMAT_YCbCr_420_P RK_FORMAT_YCrCb_420_P RK_FORMAT_YCbCr_422_P RK_FORMAT_YCrCb_422_P	width stride须4对齐，x_offset、y_offset、width、height、height stride均须2对齐

RGA2 RGA2-Lite0 RGA2-Lite1 RGA2-Lite2 RGA2-Lite3 RGA2-Enhance RGA2-Pro	4	RK_FORMAT_RGBA_8888 RK_FORMAT_BGRA_8888 RK_FORMAT_ARGB_8888 RK_FORMAT_ABGR_8888 RK_FORMAT_RGBX_8888 RK_FORMAT_BGRX_8888 RK_FORMAT_XRGB_8888 RK_FORMAT_XBGR_8888	width stride无对齐要求
		RK_FORMAT_RGBA_4444 RK_FORMAT_BGRA_4444 RK_FORMAT_ARGB_4444 RK_FORMAT_ABGR_4444 RK_FORMAT_RGBA_5551 RK_FORMAT_BGRA_5551 RK_FORMAT_ARGB_5551 RK_FORMAT_ABGR_5551 RK_FORMAT_RGB_565 RK_FORMAT_BGR_565	width stride须2对齐
		RK_FORMAT_YUYV_422 RK_FORMAT_YVYU_422 RK_FORMAT_UYVY_422 RK_FORMAT_VYUY_422 RK_FORMAT_YUYV_420 RK_FORMAT_YVYU_420 RK_FORMAT_UYVY_420 RK_FORMAT_VYUY_420	width stride须2对齐，x_offset、y_offset、width、height、height stride均须2对齐
		RK_FORMAT_RGB_888 RK_FORMAT_BGR_888 RK_FORMAT_A8	width stride须4对齐
		RK_FORMAT_YCbCr_420_SP RK_FORMAT_YCrCb_420_SP RK_FORMAT_YCbCr_422_SP RK_FORMAT_YCrCb_422_SP RK_FORMAT_YCbCr_444_SP RK_FORMAT_YCrCb_444_SP RK_FORMAT_YCbCr_420_P RK_FORMAT_YCrCb_420_P RK_FORMAT_YCbCr_422_P RK_FORMAT_YCrCb_422_P RK_FORMAT_YCbCr_400 RK_FORMAT_Y4 RK_FORMAT_Y8	width stride须4对齐，x_offset、y_offset、width、height、height stride均须2对齐
		RK_FORMAT_YCbCr_420_SP_10B RK_FORMAT_YCrCb_420_SP_10B RK_FORMAT_YCbCr_422_SP_10B RK_FORMAT_YCrCb_422_SP_10B	width stride须16对齐，x_offset、y_offset均须64对齐，width、height、height stride均须2对齐

RGA3	16	RK_FORMAT_RGBA_8888 RK_FORMAT_BGRA_8888 RK_FORMAT_ARGB_8888 RK_FORMAT_ABGR_8888 RK_FORMAT_RGBX_8888 RK_FORMAT_BGRX_8888 RK_FORMAT_XRGB_8888 RK_FORMAT_XBGR_8888	width stride须4对齐
		RK_FORMAT_RGB_565 RK_FORMAT_BGR_565	width stride须8对齐
		RK_FORMAT_YUYV_422 RK_FORMAT_YVYU_422 RK_FORMAT_UYVY_422 RK_FORMAT_VYUY_422	width stride须8对齐，x_offset、y_offset、width、height、height stride均须2对齐
		RK_FORMAT_RGB_888 RK_FORMAT_BGR_888	width stride须16对齐
		RK_FORMAT_YCbCr_420_SP RK_FORMAT_YCrCb_420_SP RK_FORMAT_YCbCr_422_SP RK_FORMAT_YCrCb_422_SP	width stride须16对齐，x_offset、y_offset、width、height、height stride均须2对齐
		RK_FORMAT_YCbCr_420_SP_10B RK_FORMAT_YCrCb_420_SP_10B RK_FORMAT_YCbCr_422_SP_10B RK_FORMAT_YCrCb_422_SP_10B	width stride须64对齐，x_offset、y_offset须4对齐，width、height、height stride均须2对齐
		FBC mode	除上述格式对齐要求外，width stride、height stride须16对齐
		TILE8*8 mode	除上述格式对齐要求外，width、height须8对齐，输入通道width stride、height stride须16对齐。

> 注：
>
> 1). 对齐要求计算公式：lcm(bpp，byte_stride * 8) / pixel_stride。
>
> 2). 当芯片平台搭载多版本硬件时，为了保证硬件利用率，librga会按最严格的对齐要求进行约束。

2. 版本说明

RGA的支持库librga.so按照一定规则更新版本号，标识着功能新增、兼容性、问题修正的更新提交，并提供几种方式查询版本号，方便开发者在使用librga.so时可以清楚的辨别当前的库文件版本是否适合于当前的开发环境。详细版本更新日志以及版本差异可以查阅源码根目录下**CHANGLOG.md**。

2.1 librga API版本说明

2.1.1 版本号格式与递增规则

2.1.1.1 版本号格式

```
major.minor.revision_[build]
```

例如：

> 1.0.0_[0]

2.1.1.2 递增规则

名称	规则
major	主版本号，当提交不向下兼容的版本。
minor	次版本号，当向下兼容的功能性API新增。
revision	修订版本号，当提交向下兼容的功能补充或致命的问题修正。
build	编译版本号，当向下兼容的问题修正。

2.1.2 版本号查询

2.1.2.1 strings命令查询：

以Android R 64位为例：

```
:/# strings vendor/lib64/librga.so |grep rga_api |grep version
rga_api version 1.0.0_[0]
```

2.1.2.2 日志打印：

当每个进程首次调用RGA API时，会打印版本号。

```
rockchiprga: rga_api version 1.0.0_[0]
```

2.1.2.3 函数接口查询

调用以下API，可以查询代码版本号、编译版本号、RGA硬件版本信息。具体使用说明可以查看 应用接口说明 章节。

```
querystring(RGA_VERSION);
```

> 字符串格式如下：
>
> RGA_api version : v1.0.0_[0]
> RGA version : RGA_2_Enhance

2.1.2.4 属性查询

该方式查询版本号仅Android系统支持，并且须已有进程调用RGA后，属性设置方生效。

```
:/# getprop |grep rga
[vendor.rga_api.version]: [1.0.0_[0]]
```

2.2 驱动版本说明

librga是基于驱动调用RGA硬件的，必须要保证驱动版本在使用的librga库的支持范围内。

2.2.1 版本号格式与递增规则

2.2.1.1 版本号格式

```
<driver_name>: v major.minor.revision
```

例如：

> RGA2 Device Driver: v2.1.0
>
> RGA multicore Device Driver: v1.2.23

2.2.1.2 递增规则

名称	规则
major	主版本号，当提交不向下兼容的版本。
minor	次版本号，当向下兼容的功能性API新增。
revision	修订版本号，当提交向下兼容的功能补充或致命的问题修正。

2.2.2 版本号查询

2.2.2.1 开机日志查询：

开机后使用以下命令查询RGA驱动初始化日志，部分早期的驱动没有打印版本号，该方法仅适用部分驱动。

```
dmesg |grep rga
```

例如：

```
[   2.382393] rga3_core0 fdb60000.rga: Adding to iommu group 2
[   2.382651] rga: rga3_core0, irq = 33, match scheduler
[   2.383058] rga: rga3_core0 hardware loaded successfully, hw_version:3.0.76831.
[   2.383121] rga: rga3_core0 probe successfully
[   2.383687] rga3_core1 fdb70000.rga: Adding to iommu group 3
[   2.383917] rga: rga3_core1, irq = 34, match scheduler
[   2.384313] rga: rga3_core1 hardware loaded successfully, hw_version:3.0.76831.
[   2.384412] rga: rga3_core1 probe successfully
[   2.384893] rga: rga2, irq = 35, match scheduler
[   2.385238] rga: rga2 hardware loaded successfully, hw_version:3.2.63318.
[   2.385257] rga: rga2 probe successfully
[   2.385455] rga_iommu: IOMMU binding successfully, default mapping core[0x1]
[   2.385586] rga: Module initialized. v1.2.23
```

其中 “v1.2.23” 便是驱动版本号。

2.2.2.2 调试节点查询

可以通过驱动调试节点查询版本号，如果没有以下节点则说明当前运行的是不支持查询的驱动版本。

- 使用默认使能CONFIG_ROCKCHIP_RGA_DEBUG_FS编译选项的kernel。

```
cat /sys/kernel/debug/rkrga/driver_version
```

- 使能ROCKCHIP_RGA_PROC_FS编译选项的kernel。

```
cat /proc/rkrga/driver_version
```

例如：

```
cat /sys/kernel/debug/rkrga/driver_version
RGA multicore Device Driver: v1.2.23
```

这里 “RGA multicore Device Driver”是指驱动名称为RGA multicore Device Driver，“v1.2.23” 是指版本为1.2.23，即说明当前驱动为1.2.23版本的RGA multicore Device Driver（通常简称multi_rga driver）驱动。

```
cat /sys/kernel/debug/rkrga/driver_version
RGA2 Device Driver: v2.1.0
```

这里 “RGA2 Device Driver” 是指驱动名称为RGA2 Device Driver，“v2.1.0” 是指版本号为2.1.0，即说明当前驱动为2.1.0版本的RGA2 Device Driver（通常简称rga2 driver）驱动。

2.3 版本对应关系

使用RGA时需要确认保证当前的运行环境是可以正常工作的，下表为常用的librga与驱动版本对应关系。

librga版本	对应驱动	硬件支持
无版本号	对应SDK内驱动	RGA1、RGA2
1.0.0 ~ 1.3.2	RGA Device Driver（kernel - 4.4及以上） RGA2 Device Driver（无版本号或v2.1.0）	RGA1、RGA2
> 1.4.0	RGA multicore Device Driver（v1.2.0及以上）	RGA2、RGA3
> 1.9.0	RGA Device Driver（kernel-4.4及以上） RGA2 Device Driver（无版本号和v2.1.0） RGA multicore Device Driver（v1.2.0及以上）	RGA1、RGA2、RGA3

3. 应用接口

RGA模块支持库为librga.so，通过对图像缓冲区结构体struct rga_info进行配置，实现相应的2D图形操作。为了获得更友好的开发体验，在此基础上进一步封装常用的2D图像操作接口。新的接口主要包含以下特点：

- 接口定义参考opencv/matlab中常用的2D图形接口定义，以减少二次开发的学习成本。
- 为消除RGA硬件版本差异带来的兼容问题，加入RGA query查询功能。查询内容主要包括版本信息，输入输出大分辨率及图像格式的支持。
- 执行图像操作之前，需要对输入输出图像缓冲区进行处理。调用wrapbuffer_T接口将输入输出图像信息填充到结构体struct rga_buffer_t，结构体中包含分辨率及图像格式等信息。
- 对于2D图像复合操作，增加improcess接口。通过传入一系列预定义的usage执行复合操作。
- 支持对单次无法完成的图像复合操作进行绑定为一个RGA图像任务，统一提交到驱动内逐个执行。

3.1 概述

该软件支持库提供以下API，异步模式仅支持C++实现。

- **querystring**： 查询获取当前芯片平台RGA硬件版本与功能支持信息，以字符串的形式返回。
- **imcheckHeader**: 校验当前使用头文件版本与librga版本差异。
- **importbuffer_T**： 将外部内存（dma_fd、虚拟地址、物理地址）导入RGA驱动内部，实现硬件快速访问物理连续/非物理连续的内存。
- **releasebuffer_handle**： 将外部buffer从RGA驱动内部解除引用与映射。
- **wrapbuffer_handle**： 快速封装图像缓冲区结构（rga_buffer_t）。
- **imbeginJob**：创建RGA图像处理任务。
- **imendJob**： 提交并执行RGA图像处理任务。
- **imcancelJob**： 取消并删除RGA图像处理任务。
- **imcopy**： 调用RGA实现快速图像拷贝操作。
- **imcopyTask**： 向RGA图像任务中添加快速图像拷贝操作。
- **imresize**： 调用RGA实现快速图像缩放操作。
- **imresizeTask**： 向RGA图像任务中添加快速图像缩放操作。
- **impyramind**： 调用RGA实现快速图像金字塔操作。
- **imcrop**： 调用RGA实现快速图像裁剪操作。
- **imcropTask**： 向RGA图像任务中添加快速图像裁剪操作。
- **imtranslate**： 调用RGA实现快速图像平移操作。
- **imtranslateTask**： 向RGA图像任务中添加快速图像平移操作。
- **imcvtcolor**： 调用RGA实现快速图像格式转换。
- **imcvtcolorTask**： 向RGA图像任务中添加快速图像格式转换。
- **imrotate**： 调用RGA实现快速图像旋转操作。
- **imrotateTask**： 向RGA图像任务中添加快速图像旋转操作。
- **imflip**： 调用RGA实现快速图像翻转操作。
- **imflipTask**： 向RGA图像任务中添加快速图像翻转操作。
- **imblend**： 调用RGA实现双通道快速图像合成操作。
- **imblendTask**： 向RGA图像任务中添加双通道快速图像合成操作。
- **imcomposite**： 调用RGA实现三通道快速图像合成操作。
- **imcompositeTask**： 向RGA图像任务中添加三通道快速图像合成操作。

- **imcolorkey**： 调用RGA实现快速图像颜色键操作。
- **imcolorkeyTask**： 向RGA图像任务中添加快速图像颜色键操作。
- **imosd**：调用RGA实现快速图像OSD字幕叠加。
- **imosdTask**：向RGA图像任务中添加快速图像OSD字幕叠加。
- **imquantize**： 调用RGA实现快速图像运算点前处理（量化）操作。
- **imquantizeTask**： 向RGA图像任务中添加快速图像运算点前处理（量化）操作。
- **imrop**： 调用RGA实现快速图像光栅操作。
- **imropTask**： 向RGA图像任务中添加快速图像光栅操作。
- **imfill**： 调用RGA实现快速图像填充操作。
- **imfillArray**： 调用RGA实现多组快速图像填充操作。
- **imfillTask**： 向RGA图像任务中添加快速图像填充操作。
- **imfillTaskArray**： 向RGA图像任务中添加多组快速图像填充操作。
- **imrectangle**： 调用RGA实现等距矩形边框快速绘制操作。
- **imrectangleArray**： 调用RGA实现多组等距矩形边框快速绘制操作。
- **imrectangleTask**： 向RGA图像任务中添加等距矩形边框快速绘制操作。
- **imrectangleTaskArray**： 向RGA图像任务中添加多组等距矩形边框快速绘制操作。
- **immakeBorder**： 调用RGA实现矩形边框快速绘制操作。
- **immosaic**：调用RGA实现快速图像马赛克遮盖。
- **immosaicArray**：调用RGA实现快速图像马赛克遮盖。
- **immosaicTask**：向RGA图像任务中添加快速图像马赛克遮盖。
- **immosaicTaskArray**：向RGA图像任务中添加快速图像马赛克遮盖。
- **improcess**： 调用RGA实现快速图像复合处理操作。
- **improcessTask**： 向RGA图像任务中添加快速图像复合处理操作。
- **imcheck**： 校验参数是否合法，以及当前硬件是否支持该操作。
- **imsync**： 用于异步模式时，同步任务完成状态。
- **imconfig**： 向当前线程上下文添加默认配置。

以下为各API在不同的环境下支持情况：

Item	API	支持语言	支持系统	librga	RGA2 驱动	Multi_RGA 驱动
common API	querystring	C++/C	Android / Linux / QNX / RT-thread	√	/	/
	imcheckHeader	C++	Android / Linux / QNX / RT-thread	⩾1.9.0	/	/
	imcheck	C++/C	Android / Linux / QNX / RT-thread	√	√	√
	imStrError	C++/C	Android / Linux / QNX / RT-thread	√	/	/
	imsync	C++/C	Android / Linux	⩾1.6.0	×	√
	imconfig	C++/C	Android / Linux	⩾1.6.0	×	√
buffer API	importbuffer_virtualaddr	C++/C	Android / Linux	⩾1.7.2	×	√
	importbuffer_physicaladdr	C++/C	Android / Linux	⩾1.7.2	×	√
	importbuffer_fd	C++/C	Android / Linux	⩾1.7.2	×	√
	releasebuffer_handle	C++/C	Android / Linux	⩾1.7.2	×	√
	wrapbuffer_handle	C++/C	Android / Linux	⩾1.7.2	/	/
	wrapbuffer_virtualaddr	C++/C	Android / Linux / QNX / RT-thread	√	/	/
	wrapbuffer_physicaladdr	C++/C	Android / Linux / QNX / RT-thread	√	/	/
	wrapbuffer_fd	C++/C	Android / Linux / QNX / RT-thread	√	/	/

Item	API	支持语言	支持系统	librga	RGA2 驱动	Multi_RGA 驱动
single process API	imcopy	C++/C	Android / Linux / QNX / RT-thread	√	√	√
	imresize	C++/C	Android / Linux / QNX / RT-thread	√	√	√
	impyramind	C++/C	Android / Linux / QNX / RT-thread	√	√	√
	imcrop	C++/C	Android / Linux / QNX / RT-thread	√	√	√
	imtranslate	C++/C	Android / Linux / QNX / RT-thread	√	√	√
	imcvtcolor	C++/C	Android / Linux / QNX / RT-thread	√	√	√
	imrotate	C++/C	Android / Linux / QNX / RT-thread	√	√	√
	imflip	C++/C	Android / Linux / QNX / RT-thread	√	√	√
	imblend	C++/C	Android / Linux / QNX / RT-thread	√	√	√
	imcomposite	C++/C	Android / Linux / QNX / RT-thread	√	√	√
	imcolorkey	C++/C	Android / Linux / QNX / RT-thread	√	√	√
	imosd	C++/C	Android / Linux / QNX / RT-thread	⩾1.8.0	×	√
	imquantize	C++/C	Android / Linux / QNX / RT-thread	√	√	√
	imrop	C++/C	Android / Linux / QNX / RT-thread	√	√	√
	imfill	C++/C	Android / Linux / QNX / RT-thread	√	√	√
	imfillArray	C++/C	Android / Linux	⩾1.9.0	×	√
	imrectangle	C++/C	Android / Linux	⩾1.9.0	×	√

Item	API	支持语言	支持系统	librga	RGA2 驱动	Multi_RGA 驱动
	imrectangleArray	C++/C	Android / Linux	⩾1.9.0	×	√
	immakeBorder	C++/C	Android / Linux	⩾1.9.0	×	√
	immosaic	C++/C	Android / Linux / QNX / RT-thread	⩾1.8.0	×	√
	immosaicArray	C++/C	Android / Linux	⩾1.9.0	×	√
	improcess	C++/C	Android / Linux / QNX / RT-thread	√	√	√
task process API	imbeginJob	C++	Android / Linux	⩾1.9.0	×	⩾1.2.25
	imendJob	C++	Android / Linux	⩾1.9.0	×	⩾1.2.25
	imcancelJob	C++	Android / Linux	⩾1.9.0	×	⩾1.2.25
	imcopyTask	C++	Android / Linux	⩾1.9.0	×	⩾1.2.25
	imresizeTask	C++	Android / Linux	⩾1.9.0	×	⩾1.2.25
	imcropTask	C++	Android / Linux	⩾1.9.0	×	⩾1.2.25
	imtranslateTask	C++	Android / Linux	⩾1.9.0	×	⩾1.2.25
	imcvtcolorTask	C++	Android / Linux	⩾1.9.0	×	⩾1.2.25
	imrotateTask	C++	Android / Linux	⩾1.9.0	×	⩾1.2.25
	imflipTask	C++	Android / Linux	⩾1.9.0	×	⩾1.2.25
	imblendTask	C++	Android / Linux	⩾1.9.0	×	⩾1.2.25
	imcompositeTask	C++	Android / Linux	⩾1.9.0	×	⩾1.2.25
	imcolorkeyTask	C++	Android / Linux	⩾1.9.0	×	⩾1.2.25
	imosdTask	C++	Android / Linux	⩾1.9.0	×	⩾1.2.25
	imquantizeTask	C++	Android / Linux	⩾1.9.0	×	⩾1.2.25
	imropTask	C++	Android / Linux	⩾1.9.0	×	⩾1.2.25
	imfillTask	C++	Android / Linux	⩾1.9.0	×	⩾1.2.25
	imfillTaskArray	C++	Android / Linux	⩾1.9.0	×	⩾1.2.25
	imrectangleTask	C++	Android / Linux	⩾1.9.0	×	⩾1.2.25
	imrectangleTaskArray	C++	Android / Linux	⩾1.9.0	×	⩾1.2.25
	immosaicTask	C++	Android / Linux	⩾1.9.0	×	⩾1.2.25
	immosaicTaskArray	C++	Android / Linux	⩾1.9.0	×	⩾1.2.25
	immosaicTask	C++	Android / Linux	⩾1.9.0	×	⩾1.2.25
	improcessTask	C++	Android / Linux	⩾1.9.0	×	⩾1.2.25

Item	API	支持语言	支持系统	librga	RGA2 驱动	Multi_RGA 驱动
expand API	importbuffer_GraphicBuffer	C++	Android	√	/	/
	importbuffer_GraphicBuffer_handle	C++	Android	√	/	/
	importbuffer_AHardwareBuffer	C++	Android	√	/	/
	wrapbuffer_GraphicBuffer	C++	Android	√	/	/
	wrapbuffer_handle（GraphicBuffer handle）	C++	Android	√	/	/
	wrapbuffer_AHardwareBuffer	C++	Android	√	/	/

> 注：
>
> （√）：支持，（×）：不支持，（/）：不相关

3.2 获取RGA 版本及支持信息

3.2.1 querystring

```
const char* querystring(int name);
```

> 查询RGA基础信息及分辨率格式等支持情况

Parameters	Description
name	RGA_VENDOR - 厂商信息 RGA_VERSION - 版本信息 RGA_MAX_INPUT - 支持的最大输入分辨率 RGA_MAX_OUTPUT - 支持的最大输出分辨率 RGA_BYTE_STRIDE - 支持的stride对齐要求 RGA_SCALE_LIMIT - 支持得缩放倍数 RGA_INPUT_FORMAT - 支持的输入格式 RGA_OUTPUT_FORMAT - 支持的输出格式 RGA_EXPECTED - 预期性能 RGA_ALL - 输出所有信息

Returns a string describing properties of RGA.

3.3 头文件版本校验

3.3.1 imcheckHeader

```
IM_API IM_STATUS imcheckHeader(im_api_version_t header_version = RGA_CURRENT_API_HEADER_VERSION);
```

> 校验当前使用头文件版本与librga版本差异。

Parameters	Description
header_version	头文件版本，通常使用宏 RGA_CURRENT_API_HEADER_VERSION即可。

Return IM_STATUS_SUCCESS on success or else negative error code.

3.4 图像缓冲区预处理

3.4.1 importbuffer_T

> 对于需要RGA处理的外部内存，可以使用importbuffer_T接口将缓冲区对应的物理地址信息映射到RGA驱动内部，并获取缓冲区相应的地址信息，方便后续的稳定、快速地调用RGA完成工作。

Parameters(T)	Data Type	Description
virtual address	void *	图像缓冲区虚拟地址
physical address	uint64_t	图像缓冲区连续的物理地址
fd	int	图像缓冲区DMA的文件描述符
GraphicBuffer handle	buffer_handle_t	图像缓冲区handle, 包含缓冲区地址，文件描述符，分辨率及格式等信息
GraphicBuffer	GraphicBuffer	android graphic buffer
AHardwareBuffer	AHardwareBuffer	chunks of memory that can be accessed by various hardware components in the system. https://developer.android.com/ndk/reference/group/a-hardware-buffer

> 不同的buffer类型调用RGA的性能是不同的，性能排序如下所示：
>
> physical address > fd > virtual address
>
> 一般推荐使用fd作为buffer类型。

```
IM_API rga_buffer_handle_t importbuffer_fd(int fd, int size);
IM_API rga_buffer_handle_t importbuffer_virtualaddr(void *va, int size);
IM_API rga_buffer_handle_t importbuffer_physicaladdr(uint64_t pa, int size);
```

Parameter	Description
fd/va/pa	**[required]** external buffer
size	**[required]** memory size

```
IM_API rga_buffer_handle_t importbuffer_fd(int fd, int width, int height, int format);
IM_API rga_buffer_handle_t importbuffer_virtualaddr(void *va, int width, int height, int format);
IM_API rga_buffer_handle_t importbuffer_physicaladdr(uint64_t pa, int width, int height, int format);
```

Parameter	Description
fd/va/pa	**[required]** external buffer
width	**[required]** pixel width stride of the image buffer
height	**[required]** pixel height stride of the image buffer
format	**[required]** pixel format of the image buffer

```
IM_API rga_buffer_handle_t importbuffer_fd(int fd, im_handle_param_t *param);
IM_API rga_buffer_handle_t importbuffer_virtualaddr(void *va, im_handle_param_t *param);
IM_API rga_buffer_handle_t importbuffer_physicaladdr(uint64_t pa, im_handle_param_t *param);
```

Parameter	Description
fd/va/pa	**[required]** external buffer
param	**[required]** configure buffer parameters

```
IM_API rga_buffer_handle_t importbuffer_GraphicBuffer_handle(buffer_handle_t hnd);
IM_API rga_buffer_handle_t importbuffer_GraphicBuffer(sp<GraphicBuffer> buf);
IM_API rga_buffer_handle_t importbuffer_AHardwareBuffer(AHardwareBuffer *buf);
```

Parameter	Description
hnd/buf	**[required]** external buffer

Returns rga_buffer_handle_t to describe the memory handle.

3.4.2 releasebuffer_handle

> 当使用外部内存调用RGA完毕后，需要通过内存句柄 handle 调用 releasebuffer_handle解除该缓冲区与RGA驱动的映射和绑定关系，并释放RGA驱动内部对应的资源。

```
IM_API IM_STATUS releasebuffer_handle(rga_buffer_handle_t handle);
```

Return IM_STATUS_SUCCESS on success or else negative error code.

3.4.3 wrapbuffer_handle

> IM2D图形库接口参数中，输入源图像及输出目标图像应支持多种类型，它主要包含内存、图像格式、图像宽高等信息。在执行相应的图像操作之前，需要先调用wrapbuffer_handle将输入输出的图像参数转化为统一的 rga_buffer_t 结构作为user API的输入参数。

```
rga_buffer_t wrapbuffer_handle(rga_buffer_handle_t handle,
                               int width,
                               int height,
                               int format,
                               int wstride = width,
                               int hstride = height);
```

Parameter	Description
handle	**[required]** RGA buffer handle
width	**[required]** pixel width of the image that needs to be processed
height	**[required]** pixel height of the image that needs to be processed
format	**[required]** pixel format
wtride	**[optional]** pixel width stride of the image
hstride	**[optional]** pixel width stride of the image

Returns a rga_buffer_t to desribe image information.

3.5 图像处理任务创建

3.5.1 imbeginJob

```
IM_API im_job_handle_t imbeginJob(uint64_t flags = 0);
```

> 创建一个RGA图像处理任务，将返回一个任务句柄，job_handle 可用于添加/删除RGA图像操作、提交/执行该任务。

Parameter	Description
flags	**[optional]** job flags

Returns im_job_handle_t to describe the job handle.

3.6 图像处理任务提交

3.6.1 imendJob

```
IM_API IM_STATUS imendJob(im_job_handle_t job_handle,
                          int sync_mode = IM_SYNC,
                          int acquire_fence_fd = 0,
                          int *release_fence_fd = NULL);
```

> 提交并执行已创建的RGA图像处理任务。完成后将自动删除当前完成的RGA图像处理任务资源。

Parameter	Description
job_handle	**[required]** job handle
sync_mode	**[optional]** wait until operation complete
acquire_fence_fd	**[optional]** Used in async mode, run the job after waiting foracquire_fence signal
release_fence_fd	**[optional]** Used in async mode, as a parameter of imsync()

Return IM_STATUS_SUCCESS on success or else negative error code.

3.7 图像处理任务取消

3.7.1 imcancelJob

```
IM_API IM_STATUS imcancelJob(im_job_handle_t job_handle);
```

> 取消并删除已创建的RGA图像处理任务。

Parameter	Description
job_handle	**[required]** job handle

Return IM_STATUS_SUCCESS on success or else negative error code.

3.8 图像拷贝

3.8.1 imcopy

```
IM_STATUS imcopy(const rga_buffer_t src,
                 rga_buffer_t dst,
                 int sync = 1,
                 int *release_fence_fd = NULL);
```

> 执行单次快速图像拷贝操作，将图像从src通道图像缓冲区拷贝到dst通道图像缓冲区上。

Parameter	Description
src	**[required]** input image
dst	**[required]** output image
sync	**[optional]** wait until operation complete
release_fence_fd	**[optional]**Used in async mode, as a parameter of imsync()

Return IM_STATUS_SUCCESS on success or else negative error code.

3.8.2 imcopyTask

```
IM_API IM_STATUS imcopyTask(im_job_handle_t job_handle,
                            const rga_buffer_t src,
                            rga_buffer_t dst);
```

> 通过job_handle向指定的任务中添加图像拷贝操作，用法和imcopy一致。

Parameter	Description
job_handle	**[required]** job handle
src	**[required]** input image
dst	**[required]** output image

Return IM_STATUS_SUCCESS on success or else negative error code.

3.9 图像缩放、图像金字塔

3.9.1 imresize

```
IM_STATUS imresize(const rga_buffer_t src,
                   rga_buffer_t dst,
                   double fx = 0,
                   double fy = 0,
                   int interpolation = INTER_LINEAR,
                   int sync = 1,
                   int *release_fence_fd = NULL);
```

> 根据不同的应用场景，可选择配置dst来描述缩放的目标图像大小，或配置缩放系数fx/fy实现缩放指定倍率的效果。同时配置dst和缩放系数fx/fy时，将采用缩放系数fx/fy计算后的结果作为目标图像大小。

> interpolation 仅硬件版本RGA1/RGA1 plus 可以支持配置，其他硬件版本RGA须查询对应TRM确认缩放算法。
>
> 注意：使用缩放系数fx/fy进行倍率缩放时，YUV等对宽高对齐有要求的格式将强制向下对齐至符合要求，使用该功能有可能会改变预期缩放效果。

Parameters	Description
src	**[required]** input image
dst	**[required]** output image; it has the size dsize (when it is non-zero) or the size computed from src.size(), fx, and fy; the type of dst is the same as of src.
fx	**[optional]** scale factor along the horizontal axis; when it equals 0, it is computed as: fx = (double) dst.width / src.width
fy	**[optional]** scale factor along the vertical axis; when it equals 0, it is computed as: fy = (double) dst.height / src.height
interpolation	**[optional]** interpolation method: INTER_NEAREST - a nearest-neighbor interpolation INTER_LINEAR - a bilinear interpolation (used by default) INTER_CUBIC - a bicubic interpolation over 4x4 pixel neighborhood
sync	**[optional]** wait until operation complete
release_fence_fd	**[optional]**Used in async mode, as a parameter of imsync()

Return IM_STATUS_SUCCESS on success or else negative error code.

3.9.2 impyramid

```
IM_STATUS impyramid (const rga_buffer_t src,
                     rga_buffer_t dst,
                     IM_SCALE direction)
```

> 金字塔缩放。根据direction 宽高同时做1/2 或者 2 倍的缩放。

Parameters	Description
src	**[required]** input image
dst	**[required]** output image;
direction	**[required]** scale mode IM_UP_SCALE —— up scale IM_DOWN_SCALE —— down scale
release_fence_fd	**[optional]**Used in async mode, as a parameter of imsync()

Return IM_STATUS_SUCCESS on success or else negative error code.

3.9.3 imresizeTask

```
IM_API IM_STATUS imresizeTask(im_job_handle_t job_handle,
                              const rga_buffer_t src,
                              rga_buffer_t dst,
                              double fx = 0,
                              double fy = 0,
                              int interpolation = 0);
```

> 通过job_handle向指定的任务中添加图像缩放操作，用法和imresize一致。

Parameters	Description
job_handle	**[required]** job handle
src	**[required]** input image
dst	**[required]** output image; it has the size dsize (when it is non-zero) or the size computed from src.size(), fx, and fy; the type of dst is the same as of src.
fx	**[optional]** scale factor along the horizontal axis; when it equals 0, it is computed as: fx = (double) dst.width / src.width
fy	**[optional]** scale factor along the vertical axis; when it equals 0, it is computed as: fy = (double) dst.height / src.height
interpolation	**[optional]** interpolation method: INTER_NEAREST - a nearest-neighbor interpolation INTER_LINEAR - a bilinear interpolation (used by default) INTER_CUBIC - a bicubic interpolation over 4x4 pixel neighborhood

Return IM_STATUS_SUCCESS on success or else negative error code.

3.10 图像裁剪

3.10.1 imcrop

```
IM_STATUS imcrop(const rga_buffer_t src,
                 rga_buffer_t dst,
                 im_rect rect,
                 int sync = 1,
                 int *release_fence_fd = NULL);
```

> 通过指定Rect 的大小区域执行图像裁剪。

Parameter	Description
src	**[required]** input image
dst	**[required]** output image
rect	**[required]** crop region x - upper-left x coordinate y - upper-left y coordinate width - region width height - region height
sync	**[optional]** wait until operation complete
release_fence_fd	**[optional]**Used in async mode, as a parameter of imsync()

Return IM_STATUS_SUCCESS on success or else negative error code.

3.10.2 imcropTask

```
IM_API IM_STATUS imcropTask(im_job_handle_t job_handle,
                            const rga_buffer_t src,
                            rga_buffer_t dst,
                            im_rect rect);
```

> 通过job_handle向指定的任务中添加图像裁剪操作，用法和imcrop一致。

Parameter	Description
job_handle	**[required]** job handle
src	**[required]** input image
dst	**[required]** output image
rect	**[required]** crop region x - upper-left x coordinate y - upper-left y coordinate width - region width height - region height

Return IM_STATUS_SUCCESS on success or else negative error code.

3.11 图像平移

3.11.1 imtranslate

```
IM_STATUS imtranslate(const rga_buffer_t src,
                      rga_buffer_t dst,
                      int x,
                      int y,
                      int sync = 1,
                      int *release_fence_fd = NULL);
```

> 对图像做平移操作，移动到（x, y）坐标位置，src和dst 宽高须一致，超出部分会被裁剪。

Parameter	Description
src	**[required]**input image
dst	**[required]** output image
x	**[required]** horizontal translation
y	**[required]** vertical translation
sync	**[optional]** wait until operation complete
release_fence_fd	**[optional]**Used in async mode, as a parameter of imsync()

Return IM_STATUS_SUCCESS on success or else negative error code.

3.11.2 imtranslateTask

```
IM_API IM_STATUS imtranslateTask(im_job_handle_t job_handle,
                                 const rga_buffer_t src,
                                 rga_buffer_t dst,
                                 int x,
                                 int y);
```

> 通过job_handle向指定的任务中添加图像平移操作，用法和imtranslate一致。

Parameter	Description
job_handle	**[required]** job handle
src	**[required]**input image
dst	**[required]** output image
x	**[required]** horizontal translation
y	**[required]** vertical translation

Return IM_STATUS_SUCCESS on success or else negative error code.

3.12 图像格式转换

3.12.1 imcvtcolor

```
IM_STATUS imcvtcolor(rga_buffer_t src,
                     rga_buffer_t dst,
                     int sfmt,
                     int dfmt,
                     int mode = IM_COLOR_SPACE_DEFAULT,
                     int sync = 1,
                     int *release_fence_fd = NULL);
```

> 格式转换功能，具体格式支持根据soc有不同请查阅图像格式支持章节。
>
> 格式可以通过rga_buffer_t 设置，也可以通过sfmt/dfmt分别配置源图像及输出图像格式，当涉及YUV/RGB色域转换时可以通过mode配置转换的色域，默认按照BT.601 limit range进行转换。

parameter	Description
src	**[required]** input image
dst	**[required]** output image
sfmt	**[optional]** source image format
dfmt	**[optional]** destination image format
Mode	**[optional]** color space mode: IM_YUV_TO_RGB_BT601_LIMIT IM_YUV_TO_RGB_BT601_FULL IM_YUV_TO_RGB_BT709_LIMIT IM_RGB_TO_YUV_BT601_LIMIT IM_RGB_TO_YUV_BT601_FULL IM_RGB_TO_YUV_BT709_LIMIT
sync	**[optional]** wait until operation complete
release_fence_fd	**[optional]**Used in async mode, as a parameter of imsync()

Return IM_STATUS_SUCCESS on success or else negative error code.

3.12.2 imcvtcolorTask

```
IM_API IM_STATUS imcvtcolorTask(im_job_handle_t job_handle,
                                rga_buffer_t src,
                                rga_buffer_t dst,
                                int sfmt,
                                int dfmt,
                                int mode = IM_COLOR_SPACE_DEFAULT);
```

> 通过job_handle向指定的任务中添加图像平移操作，用法和imcvtcolor一致。

parameter	Description
job_handle	**[required]** job handle
src	**[required]** input image
dst	**[required]** output image
sfmt	**[optional]** source image format
dfmt	**[optional]** destination image format
Mode	**[optional]** color space mode: IM_YUV_TO_RGB_BT601_LIMIT IM_YUV_TO_RGB_BT601_FULL IM_YUV_TO_RGB_BT709_LIMIT IM_RGB_TO_YUV_BT601_LIMIT IM_RGB_TO_YUV_BT601_FULL IM_RGB_TO_YUV_BT709_LIMIT

Return IM_STATUS_SUCCESS on success or else negative error code.

3.13 图像旋转

3.13.1 imrotate

```
IM_STATUS imrotate(const rga_buffer_t src,
                   rga_buffer_t dst,
                   int rotation,
                   int sync = 1,
                   int *release_fence_fd = NULL);
```

> 支持图像旋转90，180，270度。

Parameter	Description
src	**[required]** input image
dst	**[required]** output image
rotation	**[required]** rotation angle: IM_HAL_TRANSFORM_ROT_90 IM_HAL_TRANSFORM_ROT_180 IM_HAL_TRANSFORM_ROT_270
sync	**[optional]** wait until operation complete
release_fence_fd	**[optional]**Used in async mode, as a parameter of imsync()

Return IM_STATUS_SUCCESS on success or else negative error code.

3.13.2 imrotateTask

```
IM_API IM_STATUS imrotateTask(im_job_handle_t job_handle,
                              const rga_buffer_t src,
                              rga_buffer_t dst,
                              int rotation);
```

> 通过job_handle向指定的任务中添加图像旋转操作，用法和imrotate一致。

Parameter	Description
job_handle	**[required]** job handle
src	**[required]** input image
dst	**[required]** output image
rotation	**[required]** rotation angle: IM_HAL_TRANSFORM_ROT_90 IM_HAL_TRANSFORM_ROT_180 IM_HAL_TRANSFORM_ROT_270

Return IM_STATUS_SUCCESS on success or else negative error code.

3.14 图像镜像翻转

3.14.1 imfilp

```
IM_STATUS imflip (const rga_buffer_t src,
                  rga_buffer_t dst,
                  int mode,
                  int sync = 1,
                  int *release_fence_fd = NULL);
```

> 支持图像做水平、垂直镜像翻转。

Parameter	Description
src	**[required]** input image
dst	**[required]** output image
mode	**[required]** flip mode: IM_HAL_TRANSFORM_FLIP_H_V IM_HAL_TRANSFORM_FLIP_H IM_HAL_TRANSFORM_FLIP_V
sync	**[optional]** wait until operation complete
release_fence_fd	**[optional]**Used in async mode, as a parameter of imsync()

Return IM_STATUS_SUCCESS on success or else negative error code.

3.14.2 imflipTask

```
IM_API IM_STATUS imflipTask(im_job_handle_t job_handle,
                            const rga_buffer_t src,
                            rga_buffer_t dst,
                            int mode);
```

> 通过job_handle向指定的任务中添加图像镜像翻转操作，用法和imflip一致。

Parameter	Description
job_handle	**[required]** job handle
src	**[required]** input image
dst	**[required]** output image
mode	**[required]** flip mode: IM_HAL_TRANSFORM_FLIP_H_V IM_HAL_TRANSFORM_FLIP_H IM_HAL_TRANSFORM_FLIP_V

Return IM_STATUS_SUCCESS on success or else negative error code.

3.15 图像合成

3.15.1 imblend/imcomposite

```
IM_STATUS imblend(const rga_buffer_t fg_image,
                  rga_buffer_t bg_image,
                  int mode = IM_ALPHA_BLEND_SRC_OVER,
                  int sync = 1,
                  int *release_fence_fd = NULL);
```

> RGA使用A+B -> B 的图像双通道合成模式， 将前景图像（srcA通道）与背景图像（dst通道）根据配置的混合模型执行对应的Alpha叠加计算，并将合成结果输出至dst通道上，当没有配置混合模式时则默认设置为src-over模式。

```
IM_STATUS imcomposite(const rga_buffer_t fg_image,
                      const rga_buffer_t bg_image,
                      rga_buffer_t output_image,
                      int mode = IM_ALPHA_BLEND_SRC_OVER,
                      int sync = 1,
                      int *release_fence_fd = NULL);
```

> RGA使用A+B -> C 的图像三通道合成模式，将前景图像（srcA通道）与背景图像（srcB通道）根据配置的混合模型执行对应的Alpha叠加计算，并将合成结果输出至dst通道上，当没有配置混合模式时则默认设置为src-over模式。

两种图像合成模式中mode 可以配置不同的**Porter-Duff**混合模型：

> 说明Porter-Duff混合模型前，先做出如下定义：
>
> - S -标识两个混合图像中的源图像，即前景图像，为souce的缩写。
> - D -标识两个混合图像中的目标图像，即背景图像，为destination的缩写。
> - R -标识两个图像混合的结果，为result的缩写。
> - c -标识像素的颜色，即（RGBA）的RGB部分，描述图像本身色彩，为color的缩写。（注意，Porter-Duff混合模型中的色彩值（RGB）均为左乘后的结果，即原始色彩与透明度的乘积，如色彩值未左乘则需要进行预乘（Xc = Xc * Xa）操作。）
> - a -标识像素的透明度，即（RGBA）的A部分，描述图像本身的透明度，为Alpha的缩写。
> - f -标识作用于**C**或者**A**上的因子，为factor的缩写。
>
> Porter-Duff混合模型的核心公式如下：
>
> Rc = Sc * Sf + Dc * Df;
>
> 即： 结果色 = 源色彩 * 源因子 + 目标色彩 * 目标因子。
>
> Ra = Sa * Sf + Da * Df;
>
> 即： 结果透明度 = 源透明度 * 源因子 + 目标透明度 * 目标因子。

RGA支持以下几种混合模型：

> SRC:
>
> Sf = 1， Df = 0；
>
> [Rc，Ra] = [Sc，Sa]；
>
> DST:
>
> Sf = 0， Df = 1；
>
> [Rc，Ra] = [Dc，Da]；
>
> SRC_OVER：
>
> Sf = 1， Df = （1 - Sa）；
>
> [Rc，Ra] = [Sc + (1 - Sa) * Dc， Sa + (1 - Sa) * Da]；
>
> DST_OVER:
>
> Sf = (1 - Da) ， Df = 1；

> [Rc，Ra] = [Sc * (1 - Da) + Dc， Sa * (1 - Da) + Da]；

【注意】图像合成模式不支持YUV格式之间合成，imblend函数dst图像不支持YUV格式，imcomposite函数srcB图像不支持YUV格式。

Parameter	Description
fg_image	**[required]** foreground image
bg_image	**[required]** background image, when A+B->B it is also the output destination image.
output_image	**[required]** output destination image.
mode	**[optional]** blending mode: IM_ALPHA_BLEND_SRC —— SRC模式 IM_ALPHA_BLEND_DST —— DST模式 IM_ALPHA_BLEND_SRC_OVER —— SRC OVER模式 IM_ALPHA_BLEND_DST_OVER —— DST OVER模式 IM_ALPHA_BLEND_PRE_MUL —— 预乘使能，当需要预乘时须将该标识与其他模式标识进行或处理，再赋值给mode
sync	**[optional]** wait until operation complete
release_fence_fd	**[optional]**Used in async mode, as a parameter of imsync()

Return IM_STATUS_SUCCESS on success or else negative error code.

3.15.2 imblendTask/imcompositeTask

```
IM_API IM_STATUS imblendTask(im_job_handle_t job_handle,
                             const rga_buffer_t fg_image,
                             rga_buffer_t bg_image,
                             int mode = IM_ALPHA_BLEND_SRC_OVER);
```

> 通过job_handle向指定的任务中添加A+B -> B模式的图像合成操作，用法和imblend一致，当没有配置混合模式时则默认设置为src-over模式。

```
IM_API IM_STATUS imcompositeTask(im_job_handle_t job_handle,
                                 const rga_buffer_t fg_image,
                                 const rga_buffer_t bg_image,
                                 rga_buffer_t output_image,
                                 int mode = IM_ALPHA_BLEND_SRC_OVER);
```

> 通过job_handle向指定的任务中添加A+B -> C模式的图像合成操作，用法和imcomposite一致，当没有配置混合模式时则默认设置为src-over模式。

【注意】图像合成模式不支持YUV格式之间合成，imblend函数dst图像不支持YUV格式，imcomposite函数srcB图像不支持YUV格式。

Parameter	Description
job_handle	**[required]** job handle
fg_image	**[required]** foreground image
bg_image	**[required]** background image, when A+B->B it is also the output destination image.
output_image	**[required]** output destination image.
mode	**[optional]** blending mode: IM_ALPHA_BLEND_SRC —— SRC模式 IM_ALPHA_BLEND_DST —— DST模式 IM_ALPHA_BLEND_SRC_OVER —— SRC OVER模式 IM_ALPHA_BLEND_DST_OVER —— DST OVER模式 IM_ALPHA_BLEND_PRE_MUL —— 预乘使能，当需要预乘时须将该标识与其他模式标识进行或处理，再赋值给mode

Return IM_STATUS_SUCCESS on success or else negative error code.

3.16 色键（Color Key）

3.16.1 imcolorkey

```
IM_STATUS imcolorkey(const rga_buffer_t src,
                     rga_buffer_t dst,
                     im_colorkey_range range,
                     int mode = IM_ALPHA_COLORKEY_NORMAL,
                     int sync = 1,
                     int *release_fence_fd = NULL);
```

> Color Key技术是对源图像进行预处理，将符合色键过滤条件的像素的alpha分量置零，其中所述色键过滤条件为非透明的颜色值，并将预处理后的源图像与目标图像进行alpha混合模式。
>
> 该模式仅支持在源图像（src）区域的图像上针对设定的颜色范围实现Color Key功能，并叠加在目标图像（dst）区域上。

> IM_ALPHA_COLORKEY_NORMAL为正常模式，即在设定的颜色范围内的颜色作为过滤条件，在该色彩范围内的像素点Alpha分量清零，IM_ALPHA_COLORKEY_INVERTED则反之，当没有配置模式时则默认设置为IM_ALPHA_COLORKEY_NORMAL模式。

Parameters	Range	Description
max	0x0 ~ 0xFFFFFFFF	需要消去/抠取的颜色范围最大值，排列为ABGR
min	0x0 ~ 0xFFFFFFFF	需要消去/抠取的颜色范围最小值，排列为ABGR

parameter	Description
src	**[required]** input image
dst	**[required]** output image
range	**[required]** Target color range typedef struct im_colorkey_range { int max; int min; } im_colorkey_value;
Mode	**[required]** Color Key mode： IM_ALPHA_COLORKEY_NORMAL IM_ALPHA_COLORKEY_INVERTED
sync	**[optional]** wait until operation complete
release_fence_fd	**[optional]**Used in async mode, as a parameter of imsync()

Return IM_STATUS_SUCCESS on success or else negative error code.

3.16.2 imcolorkeyTask

```
IM_API IM_STATUS imcolorkeyTask(im_job_handle_t job_handle,
                                const rga_buffer_t fg_image,
                                rga_buffer_t bg_image,
                                im_colorkey_range range,
                                int mode = IM_ALPHA_COLORKEY_NORMAL);
```

> 通过job_handle向指定的任务中添加图像Color Key操作，用法和imcolorkey一致，当没有配置模式时则默认设置为IM_ALPHA_COLORKEY_NORMAL模式。

Parameters	Range	Description
max	0x0 ~ 0xFFFFFFFF	需要消去/抠取的颜色范围最大值，排列为ABGR
min	0x0 ~ 0xFFFFFFFF	需要消去/抠取的颜色范围最小值，排列为ABGR

parameter	Description
job_handle	**[required]** job handle
src	**[required]** input image
dst	**[required]** output image
range	**[required]** Target color range typedef struct im_colorkey_range { int max; int min; } im_colorkey_value;
Mode	**[required]** Color Key mode： IM_ALPHA_COLORKEY_NORMAL IM_ALPHA_COLORKEY_INVERTED

Return IM_STATUS_SUCCESS on success or else negative error code.

3.17 图像字幕叠加（OSD）

3.17.1 imosd

```
IM_API IM_STATUS imosd(const rga_buffer_t osd,
                       const rga_buffer_t bg_image,
                       const im_rect osd_rect,
                       im_osd_t *osd_config,
                       int sync = 1,
                       int *release_fence_fd = NULL);
```

> OSD（On-Screen-Display）功能，可以将文字信息叠加在视频图片上，并对字体进行亮度统计、自动反色功能。

parameter	Description
OSD	**[required]** osd block image
bg_image	**[required]** output image
osd_rect	**[required]** image region to OSD
osd_config	**[required]** OSD function config
sync	**[optional]** wait until operation complete
release_fence_fd	**[optional]**Used in async mode, as a parameter of imsync()

Return IM_STATUS_SUCCESS on success or else negative error code.

3.17.2 imosdTask

```
IM_API IM_STATUS imosdTask(im_job_handle_t job_handle,
                           const rga_buffer_t osd,
                           const rga_buffer_t bg_image,
                           const im_rect osd_rect,
                           im_osd_t *osd_config);
```

> 通过job_handle向指定的任务中添加图像OSD操作，用法和imosd一致。

parameter	Description
job_handle	**[required]** job handle
OSD	**[required]** osd block image
dst	**[required]** output image
osd_rect	**[required]** image region to OSD
osd_config	**[required]** OSD function config

Return IM_STATUS_SUCCESS on success or else negative error code.

3.18 NN运算点前处理（量化）

3.18.1 imquantize

```
IM_STATUS imquantize(const rga_buffer_t src,
                     rga_buffer_t dst,
                     rga_nn_t nn_info,
                     int sync = 1,
                     int *release_fence_fd = NULL);
```

> 目前仅RV1126 / RV1109上支持。NN运算点前处理，图像RGB 三个通道可以分开单独配置offset以及scale。

公式：

```
dst = 【(src + offset) * scale 】
```

参数范围：

Parameters	Range	Description
scale	0 ~ 3.99	10bit，从左往右，高位2个bit 表示整数部分，低位8bit表示小数部分
offset	-255 ~ 255	9bit，从左往右，高位表示符号位，地位表示0～255的偏移量

parameter	Description
src	**[required]** input image
dst	**[required]** output image
nn_info	**[required]** rga_nn_t结构体对RGB三个通道offset及scale进行单独配置 typedef struct rga_nn { int nn_flag; int scale_r; int scale_g; int scale_b; int offset_r; int offset_g; int offset_b; } rga_nn_t;
sync	**[optional]** wait until operation complete
release_fence_fd	**[optional]**Used in async mode, as a parameter of imsync()

Return IM_STATUS_SUCCESS on success or else negative error code.

3.18.2 imquantizeTask

```
IM_API IM_STATUS imquantizeTask(im_job_handle_t job_handle,
                                const rga_buffer_t src,
                                rga_buffer_t dst,
                                im_nn_t nn_info);
```

> 通过job_handle向指定的任务中添加图像量化操作，用法和imquantize一致。

parameter	Description
job_handle	**[required]** job handle
src	**[required]** input image
dst	**[required]** output image
nn_info	**[required]** rga_nn_t结构体对RGB三个通道offset及scale进行单独配置 typedef struct rga_nn { int nn_flag; int scale_r; int scale_g; int scale_b; int offset_r; int offset_g; int offset_b; } rga_nn_t;

Return IM_STATUS_SUCCESS on success or else negative error code.

3.19 图像光栅操作 ROP

3.19.1 imrop

```
IM_STATUS imrop(const rga_buffer_t src,
                rga_buffer_t dst,
                int rop_code,
                int sync = 1,
                int *release_fence_fd = NULL);
```

> 对两个图形做ROP运算

parameter	Description
src	**[required]** input image
dst	**[required]** output image
rop_code	**[required]** rop code mode IM_ROP_AND : dst = dst **AND** src; IM_ROP_OR : dst = dst **OR** src IM_ROP_NOT_DST : dst = **NOT** dst IM_ROP_NOT_SRC : dst = **NOT** src IM_ROP_XOR : dst = dst **XOR** src IM_ROP_NOT_XOR : dst = **NOT** (dst **XOR** src)
sync	**[optional]** wait until operation complete
release_fence_fd	**[optional]**Used in async mode, as a parameter of imsync()

Return IM_STATUS_SUCCESS on success or else negative error code.

3.19.2 imropTask

```
IM_API IM_STATUS imropTask(im_job_handle_t job_handle,
                           const rga_buffer_t src,
                           rga_buffer_t dst,
                           int rop_code);
```

> 通过job_handle向指定的任务中添加图像ROP运算操作，用法和imrop一致。

parameter	Description
job_handle	**[required]** job handle
src	**[required]** input image
dst	**[required]** output image
rop_code	**[required]** rop code mode IM_ROP_AND : dst = dst **AND** src; IM_ROP_OR : dst = dst **OR** src IM_ROP_NOT_DST : dst = **NOT** dst IM_ROP_NOT_SRC : dst = **NOT** src IM_ROP_XOR : dst = dst **XOR** src IM_ROP_NOT_XOR : dst = **NOT** (dst **XOR** src)

Return IM_STATUS_SUCCESS on success or else negative error code.

3.20 图像颜色填充、边框绘制

3.20.1 imfill

```
IM_STATUS imfill(rga_buffer_t dst,
                 im_rect rect,
                 int color,
                 int sync = 1,
                 int *release_fence_fd = NULL);
```

> 对图像的指定区域rect进行颜色填充。
>
> color参数按照RGBA格式填写颜色值，由高到低位分别是A，B，G，R，例如，红色：color = 0x000000ff.

【注意】填充区域rect宽高须大于或等于2

Parameter	Description
dst	**[required]** target image
rect	**[required]** image region to fill specified color width and height of rect must be greater than or equal to 2
color	**[required]** fill with color
sync	**[optional]** wait until operation complete
release_fence_fd	**[optional]**Used in async mode, as a parameter of imsync()

Return IM_STATUS_SUCCESS on success or else negative error code.

3.20.2 imfillArray

```
IM_API IM_STATUS imfillArray(rga_buffer_t dst,
                             im_rect *rect_array,
                             int array_size,
                             uint32_t color,
                             int sync = 1,
                             int *release_fence_fd = NULL);
```

> 对图像的多个区域逐个进行颜色填充。
>
> color参数按照RGBA格式填写颜色值，由高到低位分别是A，B，G，R，例如，红色：color = 0x000000ff.

【注意】填充区域rect宽高须大于或等于2

Parameter	Description
dst	**[required]** target image
rect_array	**[required]** image region array_ptr to fill specified color width and height of rect must be greater than or equal to 2
array_size	**[required]** size of region arrays.
color	**[required]** fill with color
sync	**[optional]** wait until operation complete
release_fence_fd	**[optional]**Used in async mode, as a parameter of imsync()

Return IM_STATUS_SUCCESS on success or else negative error code.

3.20.3 imfillTask

```
IM_API IM_STATUS imfillTask(im_job_handle_t job_handle,
                            rga_buffer_t dst,
                            im_rect rect,
                            uint32_t color);
```

> 通过job_handle向指定的任务中添加图像填充操作，用法和imfill一致。

【注意】填充区域rect宽高须大于或等于2

Parameter	Description
job_handle	**[required]** job handle
dst	**[required]** target image
rect	**[required]** image region to fill specified color width and height of rect must be greater than or equal to 2
color	**[required]** fill with color

Return IM_STATUS_SUCCESS on success or else negative error code.

3.20.4 imfillTaskArray

```
IM_API IM_STATUS imfillTaskArray(im_job_handle_t job_handle,
                                 rga_buffer_t dst,
                                 im_rect *rect_array,
                                 int array_size,
                                 uint32_t color);
```

> 通过job_handle向指定的任务中添加对图像多个区域进行颜色填充的操作，用法和imfillArray一致。

【注意】填充区域rect宽高须大于或等于2

Parameter	Description
job_handle	**[required]** job handle
dst	**[required]** target image
rect_array	**[required]** image region array_ptr to fill specified color width and height of rect must be greater than or equal to 2
array_size	**[required]** size of region arrays.
color	**[required]** fill with color

Return IM_STATUS_SUCCESS on success or else negative error code.

3.20.5 imrectangle

```
IM_API IM_STATUS imrectangle(rga_buffer_t dst,
                             im_rect rect,
                             uint32_t color,
                             int thickness,
                             int sync = 1,
                             int *release_fence_fd = NULL);
```

> 对图像的指定区域rect（描述为边框外径）根据指定颜色color进行绘制粗细为thickness的边框，当thickness为负时填充一个实心的矩形。
>
> color参数按照RGBA格式填写颜色值，由高到低位分别是A，B，G，R，例如，红色：color = 0x000000ff.

【注意】填充区域rect宽高须大于或等于2

Parameter	Description
dst	**[required]** target image
rect	**[required]** image region to fill specified color width and height of rect must be greater than or equal to 2
color	**[required]** fill with color
thickness	**[required]** Thickness of lines that make up the rectangle. Negative values, like -1, mean that the function has to draw a filled rectangle.
sync	**[optional]** wait until operation complete
release_fence_fd	**[optional]**Used in async mode, as a parameter of imsync()

Return IM_STATUS_SUCCESS on success or else negative error code.

3.20.6 imrectangleArray

```
IM_API IM_STATUS imrectangleArray(rga_buffer_t dst,
                                  im_rect *rect_array,
                                  int array_size,
                                  uint32_t color,
                                  int thickness,
                                  int sync = 1,
                                  int *release_fence_fd = NULL);
```

> 对图像的多个指定区域rect（描述为边框外径）逐个根据指定颜色color进行绘制粗细为thickness的边框，当thickness为负时填充一个实心的矩形。
>
> color参数按照RGBA格式填写颜色值，由高到低位分别是A，B，G，R，例如，红色：color = 0x000000ff.

【注意】填充区域rect宽高须大于或等于2

Parameter	Description
dst	**[required]** target image
rect_array	**[required]** image region array_ptr to fill specified color width and height of rect must be greater than or equal to 2
array_size	**[required]** size of region arrays.
color	**[required]** fill with color
thickness	**[required]** Thickness of lines that make up the rectangle. Negative values, like -1, mean that the function has to draw a filled rectangle.
sync	**[optional]** wait until operation complete
release_fence_fd	**[optional]**Used in async mode, as a parameter of imsync()

Return IM_STATUS_SUCCESS on success or else negative error code.

3.20.7 imrectangleTask

```
IM_API IM_STATUS imrectangleTask(im_job_handle_t job_handle,
                                 rga_buffer_t dst,
                                 im_rect rect,
                                 uint32_t color,
                                 int thickness);
```

> 通过job_handle向指定的任务中添加图像填充矩形边框操作，用法和imrectangle一致。

【注意】填充区域rect宽高须大于或等于2

Parameter	Description
job_handle	**[required]** job handle
dst	**[required]** target image
rect	**[required]** image region to fill specified color width and height of rect must be greater than or equal to 2
color	**[required]** fill with color
thickness	**[required]** Thickness of lines that make up the rectangle. Negative values, like -1, mean that the function has to draw a filled rectangle.

Return IM_STATUS_SUCCESS on success or else negative error code.

3.20.8 imrectangleTaskArray

```
IM_API IM_STATUS imrectangleTaskArray(im_job_handle_t job_handle,
                                      rga_buffer_t dst,
                                      im_rect *rect_array,
                                      int array_size,
                                      uint32_t color,
                                      int thickness);
```

> 通过job_handle向指定的任务中添加对图像绘制多个矩形边框的操作，用法和imrectangleArray一致。

【注意】填充区域rect宽高须大于或等于2

Parameter	Description
job_handle	**[required]** job handle
dst	**[required]** target image
rect_array	**[required]** image region array_ptr to fill specified color width and height of rect must be greater than or equal to 2
array_size	**[required]** size of region arrays.
color	**[required]** fill with color
thickness	**[required]** Thickness of lines that make up the rectangle. Negative values, like -1, mean that the function has to draw a filled rectangle.

Return IM_STATUS_SUCCESS on success or else negative error code.

3.20.9 immakeBorder

```
IM_API IM_STATUS immakeBorder(rga_buffer_t src,
                              rga_buffer_t dst,
                              int top,
                              int bottom,
                              int left,
                              int right,
                              int border_type,
                              int value = 0,
                              int sync = 1,
                              int acquir_fence_fd = -1,
                              int *release_fence_fd = NULL);
```

> 根据配置的top/bottom/left/right像素数，对输入图像绘制边框后，输出到输出的目标图像缓冲区上。

【注意】top/bottom/left/right值须大于或等于2

Parameter	Description
src	**[required]** input source image
dst	**[required]** output target image
top	**[required]** number of top pixels
bottom	**[required]** number of bottom pixels
left	**[required]** number of left pixels
right	**[required]** number of right pixels
border_type	**[required]** Border type IM_BORDER_CONSTANT // iiiiii abcdefgh iiiiiii with some specified value 'i' IM_BORDER_REFLECT //fedcba abcdefgh hgfedcb IM_BORDER_WRAP //cdefgh abcdefgh abcdefg
value	**[optional]** the pixel value at which the border is filled
sync	**[optional]** wait until operation complete
acquire_fence_fd	**[required]** used in async mode, run the job after waiting foracquire_fence signal
release_fence_fd	**[required]** used in async mode, as a parameter of imsync()

Return IM_STATUS_SUCCESS on success or else negative error code.

3.21 图像马赛克

3.21.1 immosaic

```
IM_API IM_STATUS immosaic(const rga_buffer_t image,
                          im_rect rect,
                          int mosaic_mode,
                          int sync = 1,
                          int *release_fence_fd = NULL);
```

> 对图像指定区域进行马赛克遮盖。

Parameter	Description
image	**[required]** target image
rect	**[required]** image region to mosaic
mosaic_mode	**[required]** set mosaic mode IM_MOSAIC_8 IM_MOSAIC_16 IM_MOSAIC_32 IM_MOSAIC_64 IM_MOSAIC_128
sync	**[optional]** wait until operation complete
release_fence_fd	**[optional]**Used in async mode, as a parameter of imsync()

Return IM_STATUS_SUCCESS on success or else negative error code.

3.21.2 immosaicArray

```
IM_API IM_STATUS immosaicArray(const rga_buffer_t image,
                               im_rect *rect_array,
                               int array_size,
                               int mosaic_mode,
                               int sync = 1,
                               int *release_fence_fd = NULL);
```

> 对图像的多个区域逐个进行马赛克遮盖。

Parameter	Description
image	**[required]** target image
rect_array	**[required]** image region array_ptr to mosaic
array_size	**[required]** size of region arrays.
mosaic_mode	**[required]** set mosaic mode IM_MOSAIC_8 IM_MOSAIC_16 IM_MOSAIC_32 IM_MOSAIC_64 IM_MOSAIC_128
sync	**[optional]** wait until operation complete
release_fence_fd	**[optional]**Used in async mode, as a parameter of imsync()

Return IM_STATUS_SUCCESS on success or else negative error code.

3.21.3 immosaicTask

```
IM_API IM_STATUS immosaicTask(im_job_handle_t job_handle,
                              const rga_buffer_t image,
                              im_rect rect,
                              int mosaic_mode);
```

> 通过job_handle向指定的任务中添加图像马赛克遮盖操作，用法和immosaic一致。

Parameter	Description
job_handle	**[required]** job handle
image	**[required]** target image
rect	**[required]** image region to mosaic
mosaic_mode	**[required]** set mosaic mode IM_MOSAIC_8 IM_MOSAIC_16 IM_MOSAIC_32 IM_MOSAIC_64 IM_MOSAIC_128

Return IM_STATUS_SUCCESS on success or else negative error code.

3.21.4 immosaicTaskArray

```
IM_API IM_STATUS immosaicTaskArray(im_job_handle_t job_handle,
                                   const rga_buffer_t image,
                                   im_rect *rect_array,
                                   int array_size,
                                   int mosaic_mode);
```

> 通过job_handle向指定的任务中添加对图像多个区域的马赛克遮盖操作，用法和immosaicArray一致。

Parameter	Description
job_handle	**[required]** job handle
image	**[required]** target image
rect_array	**[required]** image region array_ptr to mosaic
array_size	**[required]** size of region arrays.
mosaic_mode	**[required]** set mosaic mode IM_MOSAIC_8 IM_MOSAIC_16 IM_MOSAIC_32 IM_MOSAIC_64 IM_MOSAIC_128

Return IM_STATUS_SUCCESS on success or else negative error code.

3.22 图像处理

3.22.1 improcess

```
IM_STATUS improcess(rga_buffer_t src,
                    rga_buffer_t dst,
                    rga_buffer_t pat,
                    im_rect srect,
                    im_rect drect,
                    im_rect prect,
                    int acquire_fence_fd,
                    int *release_fence_fd,
                    im_opt_t *opt,
                    int usage);
```

> RGA 图像复合操作函数，其他API都是基于此API开发，improcess 可以实现更复杂的复合操作。
>
> 图像操作通过usage 的方式进行配置。

usage 参照定义：

```
typedef enum {
    /* Rotation */
    IM_HAL_TRANSFORM_ROT_90      = 1 << 0,
    IM_HAL_TRANSFORM_ROT_180     = 1 << 1,
    IM_HAL_TRANSFORM_ROT_270     = 1 << 2,
    IM_HAL_TRANSFORM_FLIP_H      = 1 << 3,
    IM_HAL_TRANSFORM_FLIP_V      = 1 << 4,
    IM_HAL_TRANSFORM_FLIP_H_V    = 1 << 5,
    IM_HAL_TRANSFORM_MASK        = 0x3f,
```

```
    /*
     * Blend
     * Additional blend usage, can be used with both source and target configs.
     * If none of the below is set, the default "SRC over DST" is applied.
     */
    IM_ALPHA_BLEND_SRC_OVER     = 1 << 6,     /* Default, Porter-Duff "SRC over DST" */
    IM_ALPHA_BLEND_SRC          = 1 << 7,     /* Porter-Duff "SRC" */
    IM_ALPHA_BLEND_DST          = 1 << 8,     /* Porter-Duff "DST" */
    IM_ALPHA_BLEND_SRC_IN       = 1 << 9,     /* Porter-Duff "SRC in DST" */
    IM_ALPHA_BLEND_DST_IN       = 1 << 10,    /* Porter-Duff "DST in SRC" */
    IM_ALPHA_BLEND_SRC_OUT      = 1 << 11,    /* Porter-Duff "SRC out DST" */
    IM_ALPHA_BLEND_DST_OUT      = 1 << 12,    /* Porter-Duff "DST out SRC" */
    IM_ALPHA_BLEND_DST_OVER     = 1 << 13,    /* Porter-Duff "DST over SRC" */
    IM_ALPHA_BLEND_SRC_ATOP     = 1 << 14,    /* Porter-Duff "SRC ATOP" */
    IM_ALPHA_BLEND_DST_ATOP     = 1 << 15,    /* Porter-Duff "DST ATOP" */
    IM_ALPHA_BLEND_XOR          = 1 << 16,    /* Xor */
    IM_ALPHA_BLEND_MASK         = 0x1ffc0,

    IM_ALPHA_COLORKEY_NORMAL    = 1 << 17,
    IM_ALPHA_COLORKEY_INVERTED  = 1 << 18,
    IM_ALPHA_COLORKEY_MASK      = 0x60000,

    IM_SYNC                     = 1 << 19,
    IM_ASYNC                    = 1 << 26,
    IM_CROP                     = 1 << 20,    /* Unused */
    IM_COLOR_FILL               = 1 << 21,
    IM_COLOR_PALETTE            = 1 << 22,
    IM_NN_QUANTIZE              = 1 << 23,
    IM_ROP                      = 1 << 24,
    IM_ALPHA_BLEND_PRE_MUL      = 1 << 25,
} IM_USAGE;
```

Parameter	Description
src	**[required]** input imageA
dst	**[required]** output image
pat	**[required]** input imageB
srect	**[required]** src crop region
drect	**[required]** dst crop region
prect	**[required]** pat crop region
acquire_fence_fd	**[required]** Used in async mode, run the job after waiting foracquire_fence signal
release_fence_fd	**[required]** Used in async mode, as a parameter of imsync()
opt	**[required]** operation options typedef struct im_opt { int color; im_colorkey_range colorkey_range; im_nn_t nn; int rop_code; int priority; int core; } im_opt_t;
usage	**[required]** image operation usage

Return IM_STATUS_SUCCESS on success or else negative error code.

3.22.2 improcessTask

```
IM_API IM_STATUS improcessTask(im_job_handle_t job_handle,
                               rga_buffer_t src,
                               rga_buffer_t dst,
                               rga_buffer_t pat,
                               im_rect srect,
                               im_rect drect,
                               im_rect prect,
                               im_opt_t *opt_ptr,
                               int usage);
```

> 通过job_handle向指定的任务中添加复合图像处理操作，用法和improcess一致。

Parameter	Description
job_handle	**[required]** job handle
src	**[required]** input imageA
dst	**[required]** output image
pat	**[required]** input imageB
srect	**[required]** src crop region
drect	**[required]** dst crop region
prect	**[required]** pat crop region
acquire_fence_fd	**[required]** Used in async mode, run the job after waiting foracquire_fence signal
release_fence_fd	**[required]** Used in async mode, as a parameter of imsync()
opt	**[required]** operation options typedef struct im_opt { int color; im_colorkey_range colorkey_range; im_nn_t nn; int rop_code; int priority; int core; } im_opt_t;
usage	**[required]** image operation usage

Return IM_STATUS_SUCCESS on success or else negative error code.

3.23 参数校验

3.23.1 imcheck

```
IM_API IM_STATUS imcheck(const rga_buffer_t src, const rga_buffer_t dst,
                         const im_rect src_rect, const im_rect dst_rect,
                         const int mode_usage);
IM_API IM_STATUS imcheck_composite(const rga_buffer_t src, const rga_buffer_t dst, const rga_buffer_t pat,
                                   const im_rect src_rect, const im_rect dst_rect, const im_rect pat_rect,
                                   const int mode_usage);
```

> 在配置完毕RGA任务参数后，可以通过该接口校验当前参数是否合法，并根据当前硬件情况判断硬件是否支持。

> 建议该接口仅在开发调试阶段使用，避免多次校验导致性能损耗。

Parameter	Description
src	**[required]** input imageA
dst	**[required]** output image
pat	**[optional]** input imageB
srect	**[required]** src crop region
drect	**[required]** dst crop region
prect	**[optional]** pat crop region
usage	**[optional]** image operation usage

Return IM_STATUS_NOERROR on success or else negative error code.

3.24 同步操作

3.24.1 imsync

```
IM_STATUS imsync(int fence_fd);
```

> RGA异步模式需要调用该接口等待操作完成，将返回的release_fence_fd作为传入参数。
>
> 其他API 将 形参sync 设置为0时，使能异步调用模式，效果相当于opengl中的 glFlush，如果进一步调用imsync 可以达到glFinish的效果。

Parameter	Description
fence_fd	**[required]** fence_fd to wait

Return IM_STATUS_SUCCESS on success or else negative error code.

3.25 线程上下文配置

3.25.1 imconfig

```
IM_STATUS  imconfig(IM_CONFIG_NAME name, uint64_t value);
```

> 通过不同的配置名对当前线程的上下文进行配置，该上下文将作为该线程的默认配置。
>
> 线程上下文的配置优先级低于接口的传参配置。如果接口传参配置未配置相关参数，则本地调用使用上下文默认配置完成本地调用；如果接口传参配置相关参数，以接口传参的配置完成本次调用。

parameter	Description
name	**[required]** context config name： IM_CONFIG_SCHEDULER_CORE —— 指定任务处理核心 IM_CONFIG_PRIORITY —— 任务优先级 IM_CHECK_CONFIG —— 校验使能
value	**[required]** config value IM_CONFIG_SCHEDULER_CORE : IM_SCHEDULER_RGA3_CORE0 IM_SCHEDULER_RGA3_CORE1 IM_SCHEDULER_RGA2_CORE0 IM_SCHEDULER_RGA3_DEFAULT IM_SCHEDULER_RGA2_DEFAULT IM_CONFIG_PRIORITY: 0 ~ 6 IM_CHECK_CONFIG: TRUE FALSE

> 注意：priority、core权限极高，操作不当可能导致系统崩溃或死锁，建议仅用于开发调试阶段，极度不建议在实际产品场景进行配置。

Return IM_STATUS_SUCCESS on success or else negative error code

4. 数据结构

本章节将详细描述应用接口中涉及的数据结构。

4.1 概述

数据结构	说明
rga_buffer_t	描述图像缓冲区信息
im_rect	描述图像实际操作区域
im_opt_t	描述图像操作选项
im_job_handle_t	RGA任务句柄
rga_buffer_handle_t	RGA驱动图像缓冲区句柄
im_handle_param_t	描述待导入图像缓冲区属性
im_context_t	当前线程默认上下文
im_nn_t	描述运算点前处理参数
im_colorkey_range	Colorkey关键色范围

4.2 详细描述

4.2.1 rga_buffer_t

- 说明

描述单一通道的图像缓冲区信息。

- 路径

im2d_api/im2d_type.h

- 定义

```
typedef struct {
    void* vir_addr;                     /* virtual address */
    void* phy_addr;                     /* physical address */
    int fd;                             /* shared fd */
    rga_buffer_handle_t handle;         /* buffer handle */

    int width;                          /* width */
    int height;                         /* height */
    int wstride;                        /* wstride */
    int hstride;                        /* hstride */
    int format;                         /* format */

    int color_space_mode;               /* color_space_mode */
    int global_alpha;                   /* global_alpha */
    int rd_mode;
} rga_buffer_t;
```

成员参数	描述
vir_addr	图像缓冲区虚拟地址。
phy_addr	图像缓冲区连续的物理地址。
fd	图像缓冲区DMA的文件描述符。
handle	导入RGA驱动的图像缓冲区对应的内存句柄。
width	图像实际操作区域的宽度，以像素为单位。
height	图像实际操作区域的高度，以像素为单位。
wstride	图像宽度的步幅，以像素为单位。
hstride	图像高度的步幅，以像素为单位。
format	图像格式。
color_space_mode	图像色域空间。
global_alpha	全局Alpha配置。
rd_mode	当前通道读取数据模式。

- 注意事项

vir_addr、phy_addr、fd、handle只需选择其一作为图像缓冲区的描述即可，如多项赋值，则只会根据默认优先级选择其一作为图像缓冲区描述，优先级如下：handle > phy_addr > fd > vir_addr。

4.2.2 im_rect

- 说明

描述单一通道的图形实际操作区域。

- 路径

im2d_api/im2d_type.h

- 定义

```
typedef struct {
    int x;        /* upper-left x */
    int y;        /* upper-left y */
    int width;    /* width */
    int height;   /* height */
} im_rect;
```

成员参数	描述
x	图像实际操作区域的起始横坐标，以像素为单位。
y	图像实际操作区域的起始纵坐标，以像素为单位。
width	图像实际操作区域的宽度，以像素为单位。
height	图像实际操作区域的高度，以像素为单位。

- 注意事项

实际操作区域不能超出图像大小，即（x + width）<= wstride， （y + height） <= hstride。

4.2.3 im_opt_t

- 说明

描述当前任务图像操作选项。

- 路径

im2d_api/im2d_type.h

- 定义

```
typedef struct im_opt {
    im_api_version_t version DEFAULT_INITIALIZER(RGA_CURRENT_API_HEADER_VERSION);
    int color;                          /* color, used by color fill */
    im_colorkey_range colorkey_range;   /* range value of color key */
    im_nn_t nn;
    int rop_code;
    int priority;
    int core;
    int mosaic_mode;
    im_osd_t osd_config;
    im_intr_config_t intr_config;
    char reserve[128];
} im_opt_t;
```

成员参数	描述
version	当前头文件版本
color	填充图像颜色配置。
colorkey_range	Colorkey关键色范围配置。
nn	运算点前处理（量化）配置。
rop_code	光栅操作ROP操作码配置。
priority	当前任务优先级配置。
core	当前任务指定硬件核心。
mosaic_mode	马赛克模式配置。
osd_config	osd字幕叠加反色/统计配置。
intr_config	提前中断模式配置。
reserve	预留位。

- 注意事项

priority、core权限极高，操作不当可能导致系统崩溃或死锁，建议仅用于开发调试阶段，极度不建议在实际产品场景进行配置。

4.2.4 im_job_handle_t

- 说明

RGA任务句柄，用于标识当前配置的RGA任务。

- 路径

im2d_api/im2d_type.h

- 定义

```
typedef uint32_t im_job_handle_t;
```

- 注意事项

配置失败后须使用 imcancelJob 释放当前任务句柄，避免内存泄漏。

4.2.5 rga_buffer_handle_t

- 说明

RGA驱动图像缓冲区句柄。

- 路径

im2d_api/im2d_type.h

- 定义

```
typedef int rga_buffer_handle_t;
```

- 注意事项

当该内存使用完毕后须使用 releasebuffer_handle 释放内存，避免内存泄漏。

4.2.6 im_handle_param_t

- 说明

描述待导入的图像缓冲区描述参数。

- 路径

im2d_api/im2d_type.h

- 定义

```
typedef struct im_handle_param {
    uint32_t width;
    uint32_t height;
    uint32_t format;
} im_handle_param_t;
```

成员参数	描述
width_stride	描述待导入图像缓冲区的水平方向步幅，以像素为单位。
height_stride	描述待导入图像缓冲区的垂直方向步幅，以像素为单位。
format	描述待导入图像缓冲区的格式。

- 注意事项

该结构描述为缓冲区内存开辟大小，如若实际开辟内存大小小于该结构配置大小，会导致 importbuffer_T 接口错误。

4.2.7 im_nn_t

- 说明

描述运算点前处理（量化）的参数。

- 路径

im2d_api/im2d_type.h

- 定义

```
typedef struct im_nn {
    int scale_r;                /* scaling factor on R channal */
    int scale_g;                /* scaling factor on G channal */
    int scale_b;                /* scaling factor on B channal */
    int offset_r;               /* offset on R channal */
    int offset_g;               /* offset on G channal */
    int offset_b;               /* offset on B channal */
} im_nn_t;
```

成员参数	描述
scale_r	red分量缩放系数。
scale_g	green分量缩放系数。
scale_b	blue分量缩放系数。
offset_r	red分量偏移值。
offset_g	green分量偏移值。
offset_b	blue分量偏移值。

- 注意事项

无

4.2.8 im_colorkey_range

- 说明

Colorkey关键色范围。

- 路径

im2d_api/im2d_type.h

- 定义

```
typedef struct {
    int max;                    /* The Maximum value of the color key */
    int min;                    /* The minimum value of the color key */
} im_colorkey_range;
```

成员参数	描述
max	关键色范围最大值。
min	关键色范围最小值。

- 注意事项

无

4.2.9 im_osd_block_t

- 说明

OSD字块描述参数配置。

- 路径

im2d_api/im2d_type.h

- 定义

```
typedef struct im_osd_block {
    int width_mode;                 // normal or different
                                    //   IM_OSD_BLOCK_MODE_NORMAL
                                    //   IM_OSD_BLOCK_MODE_DIFFERENT
    union {
        int width;                  // normal_mode block width
        int width_index;            // different_mode block width index in RAM
    };

    int block_count;                // block count

    int background_config;          // background config is bright or dark
                                    //   IM_OSD_BACKGROUND_DEFAULT_BRIGHT
                                    //   IM_OSD_BACKGROUND_DEFAULT_DARK

    int direction;                  // osd block direction
                                    //   IM_OSD_MODE_HORIZONTAL
                                    //   IM_OSD_MODE_VERTICAL

    int color_mode;                 // using src1 color or config color
                                    //   IM_OSD_COLOR_PIXEL
                                    //   IM_OSD_COLOR_EXTERNAL
    im_color_t normal_color;        // config color: normal
    im_color_t invert_color;        // config color: invert
} im_osd_block_t;
```

成员参数	描述
width_mode	字块宽度模式配置： 字块宽度相同：IM_OSD_BLOCK_MODE_NORMAL 字块宽度不同：IM_OSD_BLOCK_MODE_DIFFERENT
width/width_index	width：字块宽度相同模式时使用，标识当前task OSD字块宽度。 width_index：字块宽度不同模式时使用，标识当前使用RAM内部的宽度配置表的索引。
block_count	字块数量
background_config	文字背景亮度描述： 白底黑字：IM_OSD_BACKGROUND_DEFAULT_BRIGHT 黑底白字：IM_OSD_BACKGROUND_DEFAULT_DARK
direction	OSD方向： 垂直方向：IM_OSD_MODE_VERTICAL 水平方向：IM_OSD_MODE_HORIZONTAL
color_mode	OSD字块颜色模式： 像素颜色：IM_OSD_COLOR_PIXEL 外部指导颜色：IM_OSD_COLOR_EXTERNAL
normal_color	外部指导颜色：正常时（非反色状态）字块颜色。
invert_color	外部指导颜色：反色时字块颜色。

- 注意事项

无

4.2.10 im_osd_invert_factor_t

- 说明

OSD反色公式配置。

- 路径

im2d_api/im2d_type.h

- 定义

```
typedef struct im_osd_invert_factor {
    uint8_t alpha_max;
    uint8_t alpha_min;
    uint8_t yg_max;
    uint8_t yg_min;
    uint8_t crb_max;
    uint8_t crb_min;
} im_osd_invert_factor_t;
```

计算公式如下：

MAX(channel_min_factor, channel_max_factor - channel_value)

- 注意事项

无

4.2.11 im_osd_invert_t

- 说明

OSD反色功能配置。

- 路径

im2d_api/im2d_type.h

- 定义

```
typedef struct im_osd_invert {
    int invert_channel;         // invert channel config:
                                //   IM_OSD_INVERT_CHANNEL_NONE
                                //   IM_OSD_INVERT_CHANNEL_Y_G
                                //   IM_OSD_INVERT_CHANNEL_C_RB
                                //   IM_OSD_INVERT_CHANNEL_ALPHA
                                //   IM_OSD_INVERT_CHANNEL_COLOR
                                //   IM_OSD_INVERT_CHANNEL_BOTH
    int flags_mode;             // use external or inertnal RAM invert flags
                                //   IM_OSD_FLAGS_EXTERNAL
                                //   IM_OSD_FLAGS_INTERNAL
    int flags_index;            // flags index when using internal RAM invert flags

    uint64_t invert_flags;      // external invert flags
    uint64_t current_flags;     // current flags

    int invert_mode;            // invert use swap or factor
                                //   IM_OSD_INVERT_USE_FACTOR
```

```
                                //   IM_OSD_INVERT_USE_SWAP
    im_osd_invert_factor_t factor;

    int threash;
} im_osd_invert_t;
```

成员参数	描述
invert_channel	反色通道配置： 默认无通道反色使能：IM_OSD_INVERT_CHANNEL_NONE Y/G分量反色使能：IM_OSD_INVERT_CHANNEL_Y_G C（UV）/RB分量反色使能：IM_OSD_INVERT_CHANNEL_C_RB Alpha分量反色使能：IM_OSD_INVERT_CHANNEL_ALPHA 颜色值反色使能：IM_OSD_INVERT_CHANNEL_COLOR 即(IM_OSD_INVERT_CHANNEL_Y_G | IM_OSD_INVERT_CHANNEL_C_RB)。 全部通道反色使能：IM_OSD_INVERT_CHANNEL_BOTH 即(IM_OSD_INVERT_CHANNEL_COLOR | IM_OSD_INVERT_CHANNEL_ALPHA)。
flags_mode	反色指导模式： 内部指导：IM_OSD_FLAGS_EXTERNAL，使用内部RAM统计的反色flag指导反色功能。 外部指导：IM_OSD_FLAGS_INTERNAL，由外部指导反色功能。
flags_index	当前使用RAM内部的反色指导flag表的索引。
invert_flags	外部指导反色配置表，共64个bit。
current_flags	当前task统计的反色配置表，共64个bit，task执行结束后读取。
invert_mode	反色模式： 自定义公式：IM_OSD_INVERT_USE_FACTOR 默认公式：IM_OSD_INVERT_USE_SWAP，YUV格式对U、V分量进行交换，RGB格式对R、B分量进行交换。
factor	反色公式配置。
threash	反色阈值。

- 注意事项

无

4.2.12 im_osd_bpp2_t

- 说明

OSD BPP2 RGB格式映射配置。

- 路径

im2d_api/im2d_type.h

- 定义

```
typedef struct im_osd_bpp2 {
    uint8_t  ac_swap;       // ac swap flag
                            // 0: CA
                            // 1: AC
    uint8_t  endian_swap;   // rgba2bpp endian swap
                            // 0: Big endian
                            // 1: Little endian
    im_color_t color0;
    im_color_t color1;
} im_osd_bpp2_t;
```

成员参数	描述
ac_swap	AC排布模式。
endian_swap	大小端配置。
color0	color值为0时映射的颜色。
color1	color值为1时映射的颜色。

- 注意事项

无

4.2.13 im_osd_t

- 说明

OSD功能配置。

- 路径

im2d_api/im2d_type.h

- 定义

```
typedef struct im_osd {

  int osd_mode;                 // osd mode: statistics or auto_invert
  im_osd_block_t block_parm;    // osd block info config
  im_osd_invert_t invert_config;
  im_osd_bpp2_t bpp2_info;
} im_osd_t;
```

成员参数	描述
osd_mode	OSD模式： 统计模式：IM_OSD_MODE_STATISTICS 反色模式：IM_OSD_MODE_AUTO_INVERT
block_parm	OSD block参数配置。
invert_config	反色功能配置。
bpp2_info	bpp2-rgb格式配置。

- 注意事项

无

5. 测试用例及调试方法

> 为了让开发者更加快捷的上手上述的新接口，这里通过运行demo和对demo源码的解析以加速开发者对API的理解和运用。

5.1 测试文件说明

> 用于测试的输入与输出二进制文件需提前准备好，在/sample/sample_file目录下，存放着默认的RGBA8888格式的源图像文件可以直接使用。
>
> Android系统须将源图片存储在设备/data/目录下，Linux系统须将源图储存在设备/usr/data目录下，文件命名规则如下：

```
in%dw%d-h%d-%s.bin
out%dw%d-h%d-%s.bin

示例:
1280×720 RGBA8888的输入图像:  in0w1280-h720-rgba8888.bin
1280×720 RGBA8888的输出图像:  out0w1280-h720-rgba8888.bin
```

参数解释如下：

> 输入文件为 in ， 输出文件为 out
> --->第一个%d 是文件的索引， 一般为 0， 用于区别格式及宽高完全相同的文件
> --->第二个%d 是宽的意思， 这里的宽一般指虚宽
> --->第三个%d 是高的意思， 这里的高一般指虚高
> --->第四个%s 是格式的名字。
>
> 预置测试的部分常用图像格式如下，其他格式对应字符串名可以查看rgaUtils.cpp中查看：

format（Android）	format（Linux）	name
HAL_PIXEL_FORMAT_RGB_565	RK_FORMAT_RGB_565	"rgb565"
HAL_PIXEL_FORMAT_RGB_888	RK_FORMAT_RGB_888	"rgb888"
HAL_PIXEL_FORMAT_RGBA_8888	RK_FORMAT_RGBA_8888	"rgba8888"
HAL_PIXEL_FORMAT_RGBX_8888	RK_FORMAT_RGBX_8888	"rgbx8888"
HAL_PIXEL_FORMAT_BGRA_8888	RK_FORMAT_BGRA_8888	"bgra8888"
HAL_PIXEL_FORMAT_YCrCb_420_SP	RK_FORMAT_YCrCb_420_SP	"crcb420sp"
HAL_PIXEL_FORMAT_YCrCb_NV12	RK_FORMAT_YCbCr_420_SP	"nv12"
HAL_PIXEL_FORMAT_YCrCb_NV12_VIDEO	/	"nv12"
HAL_PIXEL_FORMAT_YCrCb_NV12_10	RK_FORMAT_YCbCr_420_SP_10B	"nv12_10"

> demo中默认的输入图像文件分辨率为1280x720，格式为RGBA8888， 则须在/data或/usr/data目录下提前准备好名为in0w1280-h720-rgba8888.bin的源图像文件，图像合成模式还须额外在/data或/usr/data目录下提前准备好名为in1w1280-h720-rgba8888.bin的源图像文件。

5.2 调试方法说明

> 运行demo后打印日志如下（以图像拷贝为例）：
>
> Android中打印日志如下：

```
# rgaImDemo --copy

librga:RGA_GET_VERSION:3.02,3.020000                    //RGA版本
ctx=0x7ba35c1520,ctx->rgaFd=3                           //RGA上下文
Start selecting mode
im2d copy ..                                            //RGA运行模式
GraphicBuffer check ok
GraphicBuffer check ok
lock buffer ok
open file ok                                            //src文件的状态，如果/data/目录下没有对应文件这里会报错
unlock buffer ok
lock buffer ok
unlock buffer ok
copying .... successfully                               //标志运行成功
open /data/out0w1280-h720-rgba8888.bin and write ok     //输出文件名以及目录
```

> Linux系统中打印日志如下：

```
# rgaImDemo --copy

librga:RGA_GET_VERSION:3.02,3.020000                    //RGA版本
ctx=0x2b070,ctx->rgaFd=3                                //RGA上下文
Rga built version:version:1.00
Start selecting mode
im2d copy ..                                            //RGA运行模式
open file                                               //src文件的状态，如果/usr/data/目录下没有对应文件这里会报错
copying .... Run successfully                           //标志运行成功
open /usr/data/out0w1280-h720-rgba8888.bin and write ok //输出文件名以及目录
```

> 当需要查看RGA运行更加详细的日志时，Android系统可以通过设置属性vendor.rga.log（Android 8及以下是sys.rga.log）来打开RGA配置log打印：

```
setprop vendor.rga.log 1        打开RGA log打印
logcat -s librga                开启并过滤log打印
setprop vendor.rga.log 0        关闭RGA log打印
```

> Linux系统中需要打开代码core/NormalRgaContext.h，将__DEBUG设置为1，重新编译即可

```
#ifdef LINUX

-#define __DEBUG 0
+#define __DEBUG 1
```

> 一般打印log如下，可将此log上传至RedMine，由RK有关工程师分析：
>
> Android系统中打印日志如下：

```
 D librga  : <<<<-------- print rgaLog -------->>>>
 D librga  : src->hnd = 0x0 , dst->hnd = 0x0
 D librga  : srcFd = 11 , phyAddr = 0x0 , virAddr = 0x0
 D librga  : dstFd = 15 , phyAddr = 0x0 , virAddr = 0x0
 D librga  : srcBuf = 0x0 , dstBuf = 0x0
 D librga  : blend = 0 , perpixelAlpha = 1
 D librga  : scaleMode = 0 , stretch = 0;
 D librga  : rgaVersion = 3.020000  , ditherEn =0
 D librga  : srcMmuFlag = 1 , dstMmuFlag = 1 , rotateMode = 0
 D librga  : <<<<-------- rgaReg -------->>>>
 D librga  : render_mode=0 rotate_mode=0
 D librga  : src:[b,0,e1000],x-y[0,0],w-h[1280,720],vw-vh[1280,720],f=0
 D librga  : dst:[f,0,e1000],x-y[0,0],w-h[1280,720],vw-vh[1280,720],f=0
 D librga  : pat:[0,0,0],x-y[0,0],w-h[0,0],vw-vh[0,0],f=0
 D librga  : ROP:[0,0,0],LUT[0]
 D librga  : color:[0,0,0,0,0]
 D librga  : MMU:[1,0,80000521]
```

```
D librga  : mode[0,0,0,0]
```

> Linux系统打印日志如下：

```
render_mode=0 rotate_mode=0
src:[0,a681a008,a68fb008],x-y[0,0],w-h[1280,720],vw-vh[1280,720],f=0
dst:[0,a6495008,a6576008],x-y[0,0],w-h[1280,720],vw-vh[1280,720],f=0
pat:[0,0,0],x-y[0,0],w-h[0,0],vw-vh[0,0],f=0
ROP:[0,0,0],LUT[0]
color:[0,0,0,0,0]
MMU:[1,0,80000521]
mode[0,0,0,0,0]
gr_color_x [0, 0, 0]
gr_color_x [0, 0, 0]
```

5.3 测试用例说明

- 测试路径位于librga源码目录下 sample/im2d_api_demo ，开发者可以根据需求修改demo的配置，建议第一次运行demo使用默认配置。
- 测试用例的编译不同的平台编译是不同的，Android平台可以使用 'mm' 命令进行编译，linux平台上在使用cmake编译librga.so时会在同目录下生成对应的测试用例。
- 将对应的测试用例编译后生成的可执行文件通过adb传入设备，添加执行权限，执行demo，查看打印log。
- 查看输出文件，检查是否与预期相符。

5.3.1 申请图像缓冲

> demo中提供了两种buffer用于RGA合成——Graphicbuffer、AHardwareBuffer。这两种buffer通过宏USE_AHARDWAREBUFFER区分。

```
目录: librga/samples/im2d_api_demo/Android.mk
（line +15）

ifeq (1,$(strip $(shell expr $(PLATFORM_SDK_VERSION) \> 25)))
/*USE_AHARDWAREBUFFER为1则使用AHardwareBuffer，为0使用Graphicbuffer*/
LOCAL_CFLAGS += -DUSE_AHARDWAREBUFFER=1
endif
```

5.3.1.1 Graphicbuffer

> 主要通过三个函数完成Graphicbuffer的初始化、填充/清空、填充rga_buffer_t结构体。

```
    /*传入src/dst的宽、高、图像格式，初始化Graphicbuffer*/
    src_buf = GraphicBuffer_Init(SRC_WIDTH, SRC_HEIGHT, SRC_FORMAT);
    dst_buf = GraphicBuffer_Init(DST_WIDTH, DST_HEIGHT, DST_FORMAT);

    /*通过枚举值FILL_BUFF/EMPTY_BUFF，执行填充/清空Graphicbuffer*/
    GraphicBuffer_Fill(src_buf, FILL_BUFF, 0);
    if(MODE == MODE_BLEND)
        GraphicBuffer_Fill(dst_buf, FILL_BUFF, 1);
    else
        GraphicBuffer_Fill(dst_buf, EMPTY_BUFF, 1);

    /*填充rga_buffer_t结构体:src、dst*/
```

```
    src = wrapbuffer_GraphicBuffer(src_buf);
    dst = wrapbuffer_GraphicBuffer(dst_buf);
```

5.3.1.2 AHardwareBuffer

> 主要通过三个函数完成AHardwareBuffer的初始化、填充/清空、填充rga_buffer_t结构体。

```
    /*传入src/dst的宽、高、图像格式，初始化AHardwareBuffer*/
    AHardwareBuffer_Init(SRC_WIDTH, SRC_HEIGHT, SRC_FORMAT, &src_buf);
    AHardwareBuffer_Init(DST_WIDTH, DST_HEIGHT, DST_FORMAT, &dst_buf);

    /*通过枚举值FILL_BUFF/EMPTY_BUFF，执行填充/清空AHardwareBuffer*/
    AHardwareBuffer_Fill(&src_buf, FILL_BUFF, 0);
    if(MODE == MODE_BLEND)
        AHardwareBuffer_Fill(&dst_buf, FILL_BUFF, 1);
    else
        AHardwareBuffer_Fill(&dst_buf, EMPTY_BUFF, 1);

    /*填充rga_buffer_t结构体:src、dst*/
    src = wrapbuffer_AHardwareBuffer(src_buf);
    dst = wrapbuffer_AHardwareBuffer(dst_buf);
```

5.3.2 查看帮助信息

> 使用如下命令获取测试用例帮助信息

```
rgaImDemo -h
rgaImDemo --help
rgaImDemo
```

> 运行成功后，便可以根据帮助信息使用demo，打印信息如下：

```
rk3399_Android10:/ # rgaImDemo
librga:RGA_GET_VERSION:3.02,3.020000
ctx=0x7864d7c520,ctx->rgaFd=3

==========================================================================================
   usage: rgaImDemo [--help/-h] [--while/-w=(time)] [--querystring/--querystring=<options>]
                    [--copy] [--resize=<up/down>] [--crop] [--rotate=90/180/270]
                    [--flip=H/V] [--translate] [--blend] [--cvtcolor]
                    [--fill=blue/green/red]
         --help/-h     Call help
         --while/w     Set the loop mode. Users can set the number of cycles by themselves.
         --querystring You can print the version or support information corresponding to the current version
of RGA according to the options.
                       If there is no input options, all versions and support information of the current
version of RGA will be printed.
                       <options>:
                       vendor           Print vendor information.
                       version          Print RGA version, and librga/im2d_api version.
                       maxinput         Print max input resolution.
                       maxoutput        Print max output resolution.
                       scalelimit       Print scale limit.
                       inputformat      Print supported input formats.
                       outputformat     Print supported output formats.
                       expected         Print expected performance.
                       all              Print all information.
         --copy        Copy the image by RGA.The default is 720p to 720p.
```

```
        --resize      resize the image by RGA.You can choose to up(720p->1080p) or down(720p->480p).
        --crop        Crop the image by RGA.By default, a picture of 300*300 size is cropped from (100,100).
        --rotate      Rotate the image by RGA.You can choose to rotate 90/180/270 degrees.

        --flip        Flip the image by RGA.You can choice of horizontal flip or vertical flip.
        --translate   Translate the image by RGA.Default translation (300,300).
        --blend       Blend the image by RGA.Default, Porter-Duff 'SRC over DST'.
        --cvtcolor    Modify the image format and color space by RGA.The default is RGBA8888 to NV12.
        --fill        Fill the image by RGA to blue, green, red, when you set the option to the corresponding
color.
==========================================================================================
```

> 所有的参数解析在目录/librga/demo/im2d_api_demo/args.cpp中。

5.3.3 循环执行demo

> 使用如下命令循环执行示例demo，循环命令必须在所有参数之前，循环次数为int型，默认每次循环间隔200ms。

```
rgaImDemo -w6 --copy
rgaImDemo --while=6 --copy
```

5.3.4 获取RGA版本及支持信息

> 使用如下命令获取版本及支持信息：

```
rgaImDemo --querystring
rgaImDemo --querystring=<options>
```

> 该命令有可选options，没有options则默认视为选择=all，可选options如下：

```
options:
    =vendor             打印厂商信息
    =version            打印版本信息
    =maxinput           打印支持的最大输入分辨率
    =maxoutput          打印支持的最大输出分辨率
    =scalelimit         打印支持的缩放倍数
    =inputformat        打印支持的输入格式
    =outputformat       打印支持的输出格式
    =expected           打印预期性能
    =all                打印所有信息
```

5.3.4.1 代码解析

> 根据main()传参决定打印出的不同信息。

```
    /*将main()传参转化为QUERYSTRING_INFO枚举值*/
    IM_INFO = (QUERYSTRING_INFO)parm_data[MODE_QUERYSTRING];
    /*打印querystring()返回的字符串，即所需要的信息*/
    printf("\n%s\n", querystring(IM_INFO));
```

5.3.5 图像缩放

> 使用如下命令进行图像缩放测试

```
rgaImDemo --resize=up
rgaImDemo --resize=down
```

> 该功能必须填入可选options，可选options如下：

```
options:
    =up                 图像分辨率放大至1920x1080
    =down               图像分辨率缩小至720x480
```

5.3.5.1 代码解析

> 根据main()传参（up/down）决定放大或是缩小，即针对不同场景，重新初始化、清空buffer，填充rga_buffer_t结构体，并将最终的存储src、dst图像数据的rga_buffer_t结构体传入imresize()。

```
    switch(parm_data[MODE_RESIZE])
    {
        /*放大图像*/
        case IM_UP_SCALE :
            /*重新初始化Graphicbuffer为分辨率1920x1080对应大小*/
            dst_buf = GraphicBuffer_Init(1920, 1080, DST_FORMAT);
            /*清空buffer*/
            GraphicBuffer_Fill(dst_buf, EMPTY_BUFF, 1);
            /*重新填充存储dst数据的rga_buffer_t结构体*/
            dst = wrapbuffer_GraphicBuffer(dst_buf);
            break;

        case IM_DOWN_SCALE :
            /*重新初始化Graphicbuffer为分辨率1920x1080对应大小*/
            dst_buf = GraphicBuffer_Init(720, 480, DST_FORMAT);
            /*清空buffer*/
            GraphicBuffer_Fill(dst_buf, EMPTY_BUFF, 1);
            /*重新填充存储dst数据的rga_buffer_t结构体*/
            dst = wrapbuffer_GraphicBuffer(dst_buf);
            break;
    }
    /*将rga_buffer_t格式的结构体src、dst传入imresize()*/
    STATUS = imresize(src, dst);
    /*根据返回的IM_STATUS枚举值打印运行状态*/
    printf("resizing .... %s\n", imStrError(STATUS));
```

5.3.6 图像裁剪

> 使用如下命令测试图像裁剪

```
rgaImDemo --crop
```

> 该功能无可选options，默认裁剪坐标LT(100,100)，RT(400,100)，LB(100,400)，RB(400,400)内的图像。

5.3.6.1 代码解析

> 将需要裁剪的大小在存储src矩形数据的im_rect结构体中赋值，并将存储src、dst图像数据的rga_buffer_t结构体传入imcrop()。

```
    /*这里通过x、y确定裁剪顶点的坐标，width、height确定裁剪区域大小*/
    src_rect.x      = 100;
    src_rect.y      = 100;
    src_rect.width  = 300;
    src_rect.height = 300;

    /*将im_rect格式的结构体src_rect与rga_buffer_t格式的结构体src、dst传入imcrop()*/
    STATUS = imcrop(src, dst, src_rect);
    /*根据返回的IM_STATUS枚举值打印运行状态*/
    printf("cropping .... %s\n", imStrError(STATUS));
```

5.3.7 图像旋转

> 使用如下命令测试图像旋转

```
rgaImDemo --rotate=90
rgaImDemo --rotate=180
rgaImDemo --rotate=270
```

> 该功能必须填入可选options，可选options如下：

```
options:
    =90                 图像旋转90° ，输出图像分辨率宽高交换
    =180                图像旋转180°，输出图像分辨率不变
    =270                图像旋转270°，输出图像分辨率宽高交换
```

5.3.7.1 代码解析

> 根据main()传参（90/180/270）决定旋转角度，并将传参转化为IM_USAGE枚举值，与存储src、dst图像数据的rga_buffer_t结构体一同传入imrotate()。

```
    /*将main()传参转化为IM_USAGE枚举值*/
    ROTATE = (IM_USAGE)parm_data[MODE_ROTATE];

    /*将标识旋转角度的IM_USAGE枚举值与rga_buffer_t格式的结构体src、dst一同传入imrotate()*/
    STATUS = imrotate(src, dst, ROTATE);
    /*根据返回的IM_STATUS枚举值打印运行状态*/
    printf("rotating .... %s\n", imStrError(STATUS));
```

5.3.8 图像镜像翻转

> 使用如下命令测试镜像翻转

```
rgaImDemo --flip=H
rgaImDemo --flip=V
```

> 该功能必须填入可选options，可选options如下：

```
options:
    =H                  图像水平镜像翻转
    =V                  图像垂直镜像翻转
```

5.3.8.1 代码解析

> 根据main函数传参（H/V）决定镜像翻转方向，并将传参转化为IM_USAGE枚举值，与存储src、dst图像数据的rga_buffer_t结构体一同传入imflip()。

```
    /*将main()传参转化为IM_USAGE枚举值*/
    FLIP = (IM_USAGE)parm_data[MODE_FLIP];

    /*将标识镜像反转方向的IM_USAGE枚举值与rga_buffer_t格式的结构体src、dst一同传入imflip()*/
    STATUS = imflip(src, dst, FLIP);
    /*根据返回的IM_STATUS枚举值打印运行状态*/
    printf("flipping .... %s\n", imStrError(STATUS));
```

5.3.9 图像颜色填充

> 使用如下命令测试颜色填充

```
rgaImDemo --fill=blue
rgaImDemo --fill=green
rgaImDemo --fill=red
```

该功能必须填入可选options，默认填充颜色在坐标LT(100,100)，RT(400,100)，LB(100,400)，RB(400,400)内的图像，可选options如下：

```
options:
    =blue               图像颜色填充为蓝色
    =green              图像颜色填充为绿色
    =red                图像颜色填充为红色
```

5.3.9.1 代码解析

> 根据main函数传参（bule/green/red）决定填充颜色，将需要填充的大小在存储dst矩形数据的im_rect结构体中赋值，并将传参转化为对应颜色的16进制数，与存储dst图像数据的rga_buffer_t结构体一同传入imfill()。

```
    /*将main()传参转化为对应颜色的16进制数*/
    COLOR = parm_data[MODE_FILL];

    /*这里通过x、y确定裁剪顶点的坐标，width、height确定填充颜色区域大小*/
    dst_rect.x      = 100;
    dst_rect.y      = 100;
    dst_rect.width  = 300;
    dst_rect.height = 300;

    /*将im_rect格式的结构体dst_rect、对应颜色的16进制数与rga_buffer_t格式的结构体src、dst一同传入imfill()*/
    STATUS = imfill(dst, dst_rect, COLOR);
    /*根据返回的IM_STATUS枚举值打印运行状态*/
    printf("filling .... %s\n", imStrError(STATUS));
```

5.3.10 图像平移

> 使用如下命令测试图像平移操作

```
rgaImDemo --translate
```

> 该功能无可选options，默认顶点（左上角坐标）平移至(300,300)，即向右平移300个像素，再向下平移300个像素。

5.3.10.1 代码解析

> 将需要平移的偏移量在存储src矩形数据的im_rect结构体中赋值，并将存储src、dst图像数据的rga_buffer_t结构体传入imtranslate()。

```
    /*这里通过x、y确定平移后图像的顶点的坐标*/
    src_rect.x = 300;
    src_rect.y = 300;

    /*将im_rect格式的结构体src_rect与rga_buffer_t格式的结构体src、dst一同传入imtranslate()*/
    STATUS = imtranslate(src, dst, src_rect.x, src_rect.y);
    /*根据返回的IM_STATUS枚举值打印运行状态*/
    printf("translating .... %s\n", imStrError(STATUS));
```

5.3.11 图像拷贝

> 使用如下命令测试图像拷贝

```
rgaImDemo --copy
```

> 该功能无可选options，默认拷贝分辨率为1280x720，格式为RGBA8888的图像。

5.3.11.1 代码解析

> 将存储src、dst图像数据的rga_buffer_t结构体传入imcopy()。

```
    /*rga_buffer_t格式的结构体src、dst传入imcopy()*/
    STATUS = imcopy(src, dst);
    /*根据返回的IM_STATUS枚举值打印运行状态*/
    printf("copying .... %s\n", imStrError(STATUS));
```

5.3.12 图像合成

> 使用如下命令测试图像合成

```
rgaImDemo --blend
```

> 该功能无可选options，默认合成模式为 IM_ALPHA_BLEND_DST 模式。

5.3.12.1 代码解析

> 将存储src、dst图像数据的rga_buffer_t结构体传入imblend()。

```
    /*rga_buffer_t格式的结构体src、dst传入imblend()*/
    STATUS = imblend(src, dst);
    /*根据返回的IM_STATUS枚举值打印运行状态*/
    printf("blending .... %s\n", imStrError(STATUS));
```

5.3.13 图像格式转换

> 使用如下命令测试图像格式转换

```
rgaImDemo --cvtcolor
```

> 该功能无可选options，默认将分辨率为1280x720的图像从RGBA8888格式转换为NV12格式。

5.3.13.1 代码解析

将需要转换的格式在rga_buffer_t的成员变量format中赋值，并将存储src、dst图像数据的rga_buffer_t结构体传入imcvtcolor()。

```
    /*将转换前后的格式赋值给对应的rga_buffer_t结构体的成员变量format*/
    src.format = HAL_PIXEL_FORMAT_RGBA_8888;
    dst.format = HAL_PIXEL_FORMAT_YCrCb_NV12;

    /*将需要转换的格式与rga_buffer_t格式的结构体src、dst一同传入imcvtcolor()*/
    STATUS = imcvtcolor(src, dst, src.format, dst.format);
    /*根据返回的IM_STATUS枚举值打印运行状态*/
    printf("cvtcolor .... %s\n", imStrError(STATUS));
```